Defense Study Book

Machine Learning & Perception ▪ LSTM (Hochreiter) ▪ rCFD and CFD

Version 2 ▪ October 2026

How this book works

Every topic has four layers. Read them in this order; the first two take about two minutes.

1. 2-minute summary – the whole topic on half a page: the one idea, the one formula, the words the examiner wants to hear.
2. Picture it – an everyday image (an analogy). You remember a picture much longer than a sentence, and it gives you words when you are nervous.
3. Board drill – the formula you must be able to write on the whiteboard. Write it by hand three times, without looking. Hands learn faster than eyes (tactile, or kinaesthetic, learning), and on the day you will stand at a board.
4. Questions – predicted exam questions. Say is what you say out loud, Write is what you put on the board, Follow-up is the next question that usually comes.

Priority labels on every question.

- ASKED BEFORE asked in a past defense (from the list of past students): learn these first.
- COURSE EXAM asked in the written course exams (LSTM, ML Advanced, Geometric DL): learn second.
- PREDICTED predicted by this book (rCFD, thesis bridges, hot topics): learn last.

Below each title, From: names the course, the chapter or unit, and where the question comes from. The examiner who asked it (or is likely to) stands on the right of the title.

Who asks what (from the list of past students):

- Prof. Klambauer: empirical risk, loss, gradient descent, backprop and the delta error, generalization. "Write the simplest version first."
- Prof. Hochreiter: delta error, quick questions (regularization, optimizers, activations, the activation from Linz), REINFORCE, and everything LSTM.
- Others (Rekabsaz, Lehner, Schlüter): Transformers and attention, RNNs, SVM, VC dimension, universal approximation.
- Prof. Pirker (elective track): rCFD, your curves, CFD basics. Chapter 7.

Study plan (suggested). Day 1: Chapters 1–2 plus all board drills. Day 2: Chapters 3–4. Day 3: Chapters 5–6. Day 4: Chapters 7–8 and the thesis. Then repeat only the summaries and board drills every morning.

Contents

1 Empirical risk minimization **3**

2 Backpropagation and the delta error **4**

3 Training: optimizers, activations, regularization **6**

4 Recurrent networks and LSTM (Prof. Hochreiter) **8**

4.1 Recurrent networks and BPTT 8

4.2 LSTM 9

4.3 Beyond LSTM: xLSTM, Mamba, Hopfield . . . 11

5 Attention and Transformers 12

6 Classical ML asked in the same slot 13

7 rCFD and CFD (Prof. Pirker) 14

8 Predicted questions and rapid-fire drill 16

8.1 Thesis questions (first 5 minutes) . . . 16

8.2 Rapid-fire (say it in one breath) . . . 17

9 Conversation chains 17

10 Hot topics and the Linz school 21

10.1 The Linz school (IML, JKU) . . . 21

10.2 Hot topics 2025–2026 . . . 22

11 ML: Advanced Techniques (Brandstetter, Holzleitner) 24

11.1 Generative models and AI for physics (Brandstetter) . . . 24

11.2 Statistical theory (Holzleitner) . . . 26

12 New 2025 courses: Geometric DL, Unsupervised, DL Architectures 28

12.1 Deep Learning: Geometric Techniques . . . 28

12.2 ML: Unsupervised Techniques . . . 29

12.3 Deep Learning: Architectures and Generative Techniques . . . 30

13 October 2026: the newest questions, derived step by step 31

1 Empirical risk minimization

2-minute summary ▪ ERM (Klambauer's favourite first question)

One idea. We want a model that is good on *future* data. We cannot measure that, because the true data distribution is unknown. So we measure the average error on the data we *have* and make it small. That is empirical risk minimization.
Four ingredients: data $Z = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$ • model $g(\mathbf{x}; \mathbf{w})$ • loss $L(y, \hat{y})$ • average + minimize.

$$R_{\text{emp}}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^{N} L(y_n, g(\mathbf{x}_n; \mathbf{w})), \qquad R_{\text{gen}}(g) = \mathbb{E}_{(\mathbf{x},y)\sim p}[L(y, g(\mathbf{x}; \mathbf{w}))].$$

Key words: i.i.d. data • unknown $p(\mathbf{x}, y)$ • generalization error • test set • cross-validation • overfitting.

Picture it. A driving school. The *generalization error* is how well you drive on any road in Austria, which nobody can test completely. The *empirical risk* is your score on the practice course. You train on the practice course (minimize R_{emp}), and the driving test on new roads (test set) estimates how you will really drive. Memorizing every bump of the practice course is overfitting.

Board drill (write it by hand 3 times). $Z = \{(\mathbf{x}_n, y_n)\}$ $\hat{y} = g(\mathbf{x}; \mathbf{w})$ $R_{\text{emp}} = \frac{1}{N}\sum_n L(y_n, g(\mathbf{x}_n; \mathbf{w}))$ $\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} R_{\text{emp}}$ $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} R_{\text{emp}}$.

Q1.1 ASKED BEFORE Write down a supervised data set, a model and the empirical risk. — Klambauer

From: DL Basic Techniques (DLNN I) · Ch. 3 Notation, Ch. 5.1 ERM · asked in past defenses
Say: A data set is N pairs of input and label, drawn i.i.d. from an unknown distribution. The model maps an input to a prediction. The loss compares prediction and label. The empirical risk is the mean loss over the training set, and training minimizes it.
Write:
$Z = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$, $\mathbf{x}_n \in \mathbb{R}^D$; $\hat{y}_n = g(\mathbf{x}_n; \mathbf{w})$; $R_{\text{emp}}(\mathbf{w}) = \frac{1}{N}\sum_n L(y_n, \hat{y}_n)$.
Follow-up: "Is this only for neural networks?" No: g can be a linear model, a random forest or a kernel machine. Only the inside of g changes.
Source: DLNN script WS24, Ch. 3 and 5.1, PDF p. 23–26, 60.

Q1.2 ASKED BEFORE Difference between empirical risk and generalization error? — Klambauer

From: DL Basic Techniques (DLNN I) · Ch. 5.2–5.3 Generalization error · asked in past defenses
Say: The generalization error is the expected loss over the true distribution: the real quality on new data. We cannot compute it because p is unknown. The empirical risk is its estimate on the training data, and it is biased low because we chose $\mathbf{w}$ on those data.
Write:
$R_{\text{gen}} = \int L(y, g(\mathbf{x}; \mathbf{w}))\, p(\mathbf{x}, y)\, \mathrm{d}\mathbf{x}\, \mathrm{d}y \approx \frac{1}{M}\sum_{m\in\text{test}} L(y_m, g(\mathbf{x}_m; \mathbf{w}))$.
Follow-up: "How do you estimate it?" Test set (law of large numbers), or L-fold cross-validation, which is almost unbiased and uses every point once for testing.
Source: DLNN script WS24, Sec. 5.2–5.3, PDF p. 60–64.

Q1.3 ASKED BEFORE Which loss functions do you know? Which properties do they need? — Klambauer

From: DL Basic Techniques (DLNN I) · Ch. 4.1 Loss functions · asked in past defenses
Say: A loss is non-negative and zero for a perfect prediction. For gradient training it must be differentiable. The 0–1 loss is not, so we train on a surrogate such as cross-entropy or the hinge loss.

Write:
Squared $\frac{1}{2}(y-\hat{y})^2$ • binary CE $-y\log\hat{y}-(1-y)\log(1-\hat{y})$ • categorical CE $-\sum_k y_k\log\hat{y}_k$ • 0–1 $\mathbb{1}[y\neq\hat{y}]$ • hinge $\max(0,1-y\,g(\mathbf{x}))$.
Follow-up: "Why squared loss for regression?" Under Gaussian noise, minimizing it equals maximizing the likelihood.
Source: DLNN script WS24, Sec. 4.1.2, Table 4.1, Eqs. (4.8)–(4.11).

Q1.4 ASKED BEFORE Show it on linear regression. — Klambauer

From: DL Basic Techniques (DLNN I) · Ch. 4.1 Linear model · asked in past defenses
Say: Linear model, squared loss. The risk is convex, so there is one minimum: closed form or gradient descent.
Write:
$R_{\text{emp}}=\frac{1}{2}\|\mathbf{y}-X\mathbf{w}\|^2,\quad \hat{\mathbf{w}}=(X^\top X)^{-1}X^\top\mathbf{y},\quad \nabla_{\mathbf{w}}R_{\text{emp}}=X^\top(X\mathbf{w}-\mathbf{y})$.
Follow-up: For one sample the gradient is $(\mathbf{w}^\top\mathbf{x}-y)\,\mathbf{x}$: *error times input*. Remember this shape; backprop has exactly the same shape, $\delta_i a_j$.
Source: DLNN script WS24, Eqs. (4.12)–(4.15), PDF p. 31.

Q1.5 COURSE EXAM Overfitting, underfitting, bias–variance, double descent. — Klambauer

From: DL Basic Techniques (DLNN I) · Ch. 5.4 Model complexity; Geometric DL exam map
Say: Too simple: underfitting, high bias. Too complex: overfitting, high variance, low training error but high test error. Classical picture: U-shaped test error. Large networks: double descent, the test error falls again after the interpolation point.
Write:
$\mathbb{E}[(y-g)^2]=\sigma^2_{\text{noise}}+\text{bias}^2+\text{variance}$.

Picture it. Darts. Bias: all darts land together but off the centre. Variance: the darts scatter around the centre.

Source: DLNN script WS24, Sec. 5.4, PDF p. 65.

2 Backpropagation and the delta error

2-minute summary ▪ Backprop (asked by Klambauer AND Hochreiter)

One idea. To learn, every weight needs its gradient. Backprop gets all of them with one forward pass and one backward pass. The trick is the *delta error*: how much the loss changes when the pre-activation of one neuron changes. Deltas are computed at the output and passed *down* layer by layer.
Three lines to know by heart:

$$\underbrace{\delta_i=\frac{\partial L}{\partial s_i}}_{\text{definition}}\qquad \underbrace{\frac{\partial L}{\partial w_{ij}}=\delta_i\,a_j}_{\text{gradient}}\qquad \underbrace{\delta_j=f'(s_j)\sum_i\delta_i\,w_{ij}}_{\text{recursion}}$$

Output with a matching pair (softmax + cross-entropy, sigmoid + BCE, linear + squared): $\boldsymbol{\delta}^{[L]}=\hat{\mathbf{y}}-\mathbf{y}$.
Key words: forward pass stores s and a • chain rule • delta • outer product • $O(W)$ • vanishing / exploding gradients (Hochreiter 1991).

Picture it. A football team that lost 3–0. The coach (loss) starts at the goal that was conceded and walks *backwards* through the team. Each player's share of the blame (delta) is: the blame of the players he passed to, weighted by how strongly he passed to them (w_{ij}), times how awake he was himself ($f'(s_j)$). A tired player ($f' \approx 0$, saturated sigmoid) passes no blame on: that is the vanishing gradient.

Board drill (write it by hand 3 times). Forward: $s_i = \sum_j w_{ij} a_j$, $a_i = f(s_i)$. Backward: $\delta_k = \hat{y}_k - y_k$ (output), $\delta_j = f'(s_j) \sum_i \delta_i w_{ij}$ (hidden). Update: $\Delta w_{ij} = -\eta\, \delta_i\, a_j$. Matrix form: $\boldsymbol{\delta}^{[l-1]\top} = \boldsymbol{\delta}^{[l]\top} W^{[l]}\, \mathrm{diag}(f'(\mathbf{s}^{[l-1]}))$, $\partial L / \partial W^{[l]} = \mathbf{a}^{[l-1]} \boldsymbol{\delta}^{[l]\top}$.

Q2.1 ASKED BEFORE Derive the gradient for one weight. Define the delta. — Klambauer, Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 4.6 Backpropagation · asked in past defenses

Say: By the chain rule the loss depends on w_{ij} only through s_i. The derivative of s_i with respect to w_{ij} is just a_j. What remains, $\partial L / \partial s_i$, we call the delta.

Write:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial s_i} \frac{\partial s_i}{\partial w_{ij}} = \delta_i\, a_j, \quad \text{since } s_i = \textstyle\sum_j w_{ij} a_j \Rightarrow \partial s_i / \partial w_{ij} = a_j.$$

Follow-up: "Interpret it." Gradient = error signal above × activity below. The same shape as linear regression: error × input.

Source: DLNN script WS24, Eqs. (4.63)–(4.68), PDF p. 45.

Q2.2 ASKED BEFORE Derive the recursive formula for the delta of a hidden unit. — Klambauer, Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 4.6–4.7 Delta recursion · asked in past defenses

Say: A hidden unit j influences the loss through all units i it feeds. So I sum over them with the chain rule.

Write:

$$\delta_j = \frac{\partial L}{\partial s_j} = \textstyle\sum_i \dfrac{\partial L}{\partial s_i} \dfrac{\partial s_i}{\partial s_j} = \sum_i \delta_i\, w_{ij}\, f'(s_j), \quad \text{because } s_i = \textstyle\sum_j w_{ij} f(s_j).$$

Follow-up: Past students got stuck on indices; Prof. Hochreiter helped. Keep i = layer above, j = layer below, w_{ij} = from j to i.

Source: DLNN script WS24, Eq. (4.71), PDF p. 46.

Q2.3 ASKED BEFORE Delta at the output for softmax + cross-entropy? — Hochreiter

From: DL Basic Techniques (DLNN I) · Table 4.1 Output deltas · asked in past defenses

Say: Simply prediction minus label, $\boldsymbol{\delta} = \hat{\mathbf{y}} - \mathbf{y}$. The derivative of the log inside the cross-entropy cancels the derivative of the softmax.

Write:

$L = -\sum_k y_k \log \hat{y}_k$, $\hat{y}_k = \frac{e^{s_k}}{\sum_m e^{s_m}}$, $\frac{\partial \hat{y}_k}{\partial s_i} = \hat{y}_k(\mathbb{1}_{ki} - \hat{y}_i) \Rightarrow \frac{\partial L}{\partial s_i} = -\sum_k y_k(\mathbb{1}_{ki} - \hat{y}_i) = \hat{y}_i - y_i$ (using $\sum_k y_k = 1$).

Follow-up: Same result for sigmoid + BCE and linear + squared loss: that is why these pairs belong together.

Source: DLNN script WS24, Table 4.1, Eq. (4.78).

Q2.4 ASKED BEFORE Vanishing and exploding gradients: where do they come from? — Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 4.7 Vanishing gradients · asked in past defenses

Say: Each layer multiplies the delta by $W^{[l]}\mathrm{diag}(f'(\mathbf{s}))$. After many layers this is a product of many matrices, so the norm shrinks or grows exponentially. For the sigmoid $|f'| \le 0.25$, so it shrinks. Sepp Hochreiter showed this in 1991 in Linz.

Write:

$\|\boldsymbol{\delta}^{[l-k]}\| \lesssim \left(\max |f'| \cdot \|W\|\right)^k \|\boldsymbol{\delta}^{[l]}\|$.

Follow-up: "Remedies?" ReLU / SELU, careful initialization (keep the Jacobian norm near 1), normalization, residual connections, LSTM for time.
Source: DLNN script WS24, box after Eq. (4.84), PDF p. 52–53.

Q2.5 ASKED BEFORE How is a neural network built? What is one neuron? — Klambauer

From: DL Basic Techniques (DLNN I) · Ch. 3–4 Neuron, MLP · asked in past defenses
Say: A neuron takes a weighted sum of its inputs plus a bias, the pre-activation, and applies a non-linear activation. Neurons form layers; layers are stacked; the last layer gives the output.
Write:
$s = \mathbf{w}^\top \mathbf{a} + b$, $a = f(s)$; layer: $\mathbf{a}^{[l]} = f(W^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]})$.
Follow-up: "And without activation functions?" Then all layers collapse into one linear map $W^{[L]} \cdots W^{[1]} = W$. It cannot even solve XOR.
Source: DLNN script WS24, Eqs. (3.2)–(3.3), Sec. 4.5.

3 Training: optimizers, activations, regularization

2-minute summary ▪ The rapid-fire block (Hochreiter asks these fast)

Optimizers: GD, SGD, momentum, AdaGrad, RMSprop, Adam. Activations: sigmoid, tanh, ReLU, Leaky ReLU, ELU, SELU. From Linz: ELU (Clevert, Unterthiner, Hochreiter) and SELU (Klambauer et al. 2017). Regularization: weight decay, dropout, early stopping, data augmentation, noise, Flat Minimum Search (Hochreiter). Output layer: linear (regression), sigmoid (binary), softmax (multi-class). Why mini-batches: good gradient estimate, fast on GPU, useful noise.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla R_{\text{emp}}, \qquad \text{Adam: } \mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{v}}} + \epsilon}, \qquad \text{SELU: } \lambda \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \le 0 \end{cases}$$

with $\lambda \approx 1.0507$, $\alpha \approx 1.6733$.

Picture it. Walking down a foggy mountain. Gradient descent: look at the slope under your feet and step downhill. Momentum: a heavy ball that keeps rolling through small bumps. Adam: shoes that adjust their step to how steep and how noisy each direction is. Mini-batch: asking a few people for the way instead of everybody (too slow) or one person (too noisy).

Board drill (write it by hand 3 times). GD $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla R_{\text{emp}}$ • momentum $\mathbf{v} \leftarrow \mu \mathbf{v} - \eta \nabla R_{\text{emp}}$, $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$ • weight decay $R_{\text{emp}} + \frac{\lambda}{2}\|\mathbf{w}\|^2$ • dropout: keep unit with prob. $1 - p$ • batch norm $\hat{s} = (s - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$, then $\gamma \hat{s} + \beta$.

Q3.1 ASKED BEFORE Which activation function comes from Linz? What does it do? — Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 10.4 Self-normalization, Ch. 11 Activations · asked in past defenses
Say: SELU, the scaled exponential linear unit, from Klambauer, Unterthiner, Mayr and Hochreiter (2017); and before it ELU. SELU makes networks *self-normalizing*: mean and variance of the activations are pulled to $(0, 1)$ layer by layer, without batch normalization.
Write:
The map of (mean, variance) from one layer to the next has the fixed point $(0, 1)$; its Jacobian there has spectral norm $0.7877 < 1$, so the fixed point is stable (a contraction).

Picture it. A thermostat: whatever the temperature, the layer pushes it back to the set point.

Source: DLNN script WS24, Sec. 10.4, PDF p. 137–141.

Q3.2 ASKED BEFORE Which optimizers do you know? Are there methods other than gradient descent? Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 7 Optimizers · asked in past defenses

Say: GD, SGD, momentum, AdaGrad, RMSprop, Adam. Other methods: closed form (linear models), IRLS, RProp and Quickprop, second-order methods (Newton, Gauss–Newton, Levenberg–Marquardt, conjugate gradients, natural gradient), and gradient-free search (evolutionary algorithms).

Source: DLNN script WS24, Ch. 7, PDF p. 85–101; SS25 script "Higher Order Optimization".

Q3.3 ASKED BEFORE Which regularization techniques do you know? Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 8 Regularization · asked in past defenses

Say: Weight decay, dropout, early stopping, data augmentation, input and weight noise, weight sharing, multi-task learning, pruning (Optimal Brain Damage and Surgeon), and Flat Minimum Search, which looks for wide minima that generalize better.

Write:

Weight decay gradient: $\nabla R_{\text{emp}} + \lambda \mathbf{w}$ (the weights shrink each step).

Follow-up: "Why does dropout work?" It trains an ensemble of thinned networks and stops units from co-adapting.

Source: DLNN script WS24, Ch. 8, PDF p. 103–119.

Q3.4 ASKED BEFORE How can you do feature selection? Hochreiter

From: ML: Supervised Techniques · feature selection · asked in past defenses

Say: Three families. Filter: rank features by a score like correlation or mutual information. Wrapper: search subsets with the model, e.g. forward or backward selection. Embedded: the model selects while it trains, e.g. L1 regularization (Lasso) pushes weights to exactly zero.

Q3.5 PREDICTED Why initialize randomly? What does good initialization do? Klambauer

From: DL Basic Techniques (DLNN I) · Ch. 9 Initialization · predicted

Say: Equal weights make all units in a layer learn the same thing, so we break the symmetry with random weights. The scale should keep the variance of activations forward and of deltas backward constant, so nothing vanishes or explodes.

Write:

$\text{Var}(w) = 1/n_{\text{in}}$ (LeCun), $2/n_{\text{in}}$ (He, ReLU), $2/(n_{\text{in}} + n_{\text{out}})$ (Glorot).

Source: DLNN script WS24, Ch. 9, PDF p. 123–129.

Q3.6 ASKED BEFORE Universal approximation theorem. Lehner

From: ML: Supervised Techniques (Lehner) unit 6; DLNN II theory chapter · asked in past defenses

Say: One hidden layer with finitely many units and a non-polynomial activation can approximate any continuous function on a compact set arbitrarily well. It says such a network exists; it does not say how many units, nor that gradient descent finds it.

Source: DLNN I+II script SS25, "Deep Learning theory", PDF p. 321.

Q3.7 PREDICTED CNNs: why do they work for images? What is ResNet? Klambauer

From: DL Basic Techniques (DLNN I) · Ch. 6 CNNs; DL Architectures (DLNN II) · ResNet · predicted

Say: Local filters with shared weights slide over the image: few parameters, translation equivariance. Pooling reduces size and adds invariance. Depth grows the receptive field. ResNet adds identity skips, $\mathbf{a}^{[l+1]} = \mathbf{a}^{[l]} + F(\mathbf{a}^{[l]})$, so the Jacobian is $I + \partial F$ and gradients pass through hundreds of layers.

Write:
Output size $\lfloor (n + 2p - k)/s \rfloor + 1$.
Source: DLNN script WS24, Ch. 6; SS25 script PDF p. 176, 323.

4 Recurrent networks and LSTM (Prof. Hochreiter)

This is Prof. Hochreiter's own field: he found the vanishing gradient (1991) and invented LSTM with Schmidhuber (1997). Expect him to go deep and to enjoy it. Notation of his lecture: input $\mathbf{x}(t)$, pre-activation $\mathbf{s}(t)$, activation $\mathbf{a}(t)$ or hidden state $\mathbf{h}(t)$, memory cell $\mathbf{c}(t)$, input weights W, recurrent weights R, output weights V.

4.1 Recurrent networks and BPTT

2-minute summary ▪ RNN in two minutes

One idea. An RNN reads a sequence one step at a time and keeps a hidden state: a summary of everything seen so far. The *same* weights are used at every step, so it works for any length. Training = backpropagation through time (BPTT): unroll the net over time and backprop through the unrolled graph.

$$\mathbf{s}(t) = W\mathbf{x}(t) + R\,\mathbf{a}(t-1), \qquad \mathbf{a}(t) = f(\mathbf{s}(t)), \qquad \hat{\mathbf{y}}(t) = V\mathbf{a}(t), \qquad L = \sum_{t=1}^{T} L(t).$$

$$\boldsymbol{\delta}(t) = \operatorname{diag}(f'(\mathbf{s}(t)))\Big(V^\top \frac{\partial L(t)}{\partial \hat{\mathbf{y}}(t)} + R^\top \boldsymbol{\delta}(t+1)\Big),$$

$$\frac{\partial L}{\partial R} = \sum_t \boldsymbol{\delta}(t)\,\mathbf{a}(t-1)^\top, \qquad \frac{\partial L}{\partial W} = \sum_t \boldsymbol{\delta}(t)\,\mathbf{x}(t)^\top.$$

Key words: weight sharing in time • Turing complete • BPTT • truncated BPTT • RTRL • vanishing gradient in time.

Picture it. Reading a novel. You do not remember every word, you keep a running summary in your head (the hidden state) and update it with each new sentence, using the same reading skill (the same weights) on every page. BPTT is re-reading the book backwards to find which pages caused your wrong guess about the murderer.

Board drill (write it by hand 3 times). $\mathbf{s}(t) = W\mathbf{x}(t) + R\mathbf{a}(t-1)$, $\mathbf{a}(t) = f(\mathbf{s}(t))$, $\hat{\mathbf{y}}(t) = V\mathbf{a}(t)$; $\boldsymbol{\delta}(t) = \operatorname{diag}(f'(\mathbf{s}(t)))\,(V^\top \partial L(t)/\partial \hat{\mathbf{y}}(t) + R^\top \boldsymbol{\delta}(t+1))$; $\partial L/\partial R = \sum_t \boldsymbol{\delta}(t)\mathbf{a}(t-1)^\top$. Note: $\boldsymbol{\delta}(t+1)$, the future, not $t-1$. Gradient uses $\mathbf{a}(t-1)$, not $\mathbf{s}(t-1)$.

Q4.1 COURSE EXAM Which recurrent architectures do you know? Hochreiter

From: LSTM & Transformers (Hochreiter) · RNN architectures (Jordan, Elman, NARX, TDNN) · course exam
Say: Jordan network: feeds the *output* of $t-1$ back as input. Elman network: feeds the *hidden* state back, originally with fixed recurrent weights and trained locally in time. Fully recurrent network: trainable recurrent matrix R, every hidden unit sees all others. NARX: shortcuts to outputs further in the past (delays), which fights vanishing gradients only for a finite number of steps. Time-delay networks (TDNN): a fixed window, basically a 1D convolution. Then LSTM and GRU with gates.
Follow-up: "Jordan training trick?" Teacher forcing: during training feed the true label of $t-1$ instead of the prediction.

Source: LSTM lecture WS24, `Exam_LSTM summary.pdf`.

Q4.2 COURSE EXAM BPTT, truncated BPTT and RTRL: compare them. Hochreiter

From: LSTM & Transformers (Hochreiter) · BPTT, truncated BPTT, RTRL · course exam

Say: BPTT: forward through the whole sequence, then backward; exact gradient; memory grows linearly with the length T; cost $O(T \cdot W)$. Truncated BPTT: backprop only k steps into the past; cheaper, but the gradient is an approximation and long dependencies are cut. RTRL: carries the derivative of the state with respect to all weights forward in time, so it is online and memory does not grow with T, but it is very expensive: $O(N^3)$ numbers stored and $O(N^4)$ operations per step for N hidden units.

Write:

RTRL recursion: $\dfrac{\partial \mathbf{s}(t)}{\partial w} = \dfrac{\partial \mathbf{s}(t)}{\partial w}\Big|_{\text{direct}} + \dfrac{\partial \mathbf{s}(t)}{\partial \mathbf{s}(t-1)} \dfrac{\partial \mathbf{s}(t-1)}{\partial w}.$

Picture it. BPTT is watching the whole match recording afterwards. RTRL is a coach who updates his notes live during the match: no recording needed, but he must write very fast.

Source: LSTM lecture WS24, BPTT/RTRL exam questions.

Q4.3 COURSE EXAM Why can a plain RNN not learn long-term dependencies? Hochreiter

From: LSTM & Transformers (Hochreiter) · vanishing gradient in time · course exam

Say: The delta is multiplied at every time step by the recurrent Jacobian $R^\top \mathrm{diag}(f'(\mathbf{s}))$. Over k steps this is a product of k factors, so it decays (or explodes) exponentially in k. After 10–20 steps the error from the past is practically zero.

Write:

$\left\| \dfrac{\partial \mathbf{a}(t)}{\partial \mathbf{a}(t-k)} \right\| = \left\| \prod_{\tau=t-k+1}^{t} \mathrm{diag}(f'(\mathbf{s}(\tau)))R \right\| \leq \left(\max |f'| \, \|R\|\right)^k.$

Follow-up: "Gradient clipping?" Only fixes *exploding*: it rescales the gradient norm to a threshold, keeps the direction. It does nothing for vanishing.

4.2 LSTM

2-minute summary ▪ LSTM in two minutes

One idea. Give the network a *memory cell* that is updated by *adding*, not by multiplying through a squashing function. Then the error can flow back through time unchanged: the *constant error carousel* (CEC). Gates, small sigmoid switches between 0 and 1, decide what to write (input gate), what to keep (forget gate) and what to show (output gate).

$$
\begin{aligned}
\mathbf{z}(t) &= \tanh(W_z \mathbf{x}(t) + R_z \mathbf{h}(t-1) + \mathbf{b}_z) && \text{cell input} \\
\mathbf{i}(t) &= \sigma(W_i \mathbf{x}(t) + R_i \mathbf{h}(t-1) + \mathbf{b}_i) && \text{input gate} \\
\mathbf{f}(t) &= \sigma(W_f \mathbf{x}(t) + R_f \mathbf{h}(t-1) + \mathbf{b}_f) && \text{forget gate} \\
\mathbf{o}(t) &= \sigma(W_o \mathbf{x}(t) + R_o \mathbf{h}(t-1) + \mathbf{b}_o) && \text{output gate} \\
\mathbf{c}(t) &= \mathbf{f}(t) \odot \mathbf{c}(t-1) + \mathbf{i}(t) \odot \mathbf{z}(t) && \text{memory update} \\
\mathbf{h}(t) &= \mathbf{o}(t) \odot \tanh(\mathbf{c}(t)) && \text{output}
\end{aligned}
$$

Why it works: $\dfrac{\partial \mathbf{c}(t)}{\partial \mathbf{c}(t-1)} = \mathrm{diag}(\mathbf{f}(t))$, which is $\approx I$ when the forget gate is open. The original 1997 LSTM had no forget gate: $\mathbf{c}(t) = \mathbf{c}(t-1) + \mathbf{i} \odot \mathbf{z}$, derivative exactly I. The forget gate was added by Gers, Schmidhuber and Cummins (2000).

Picture it. A conveyor belt (the cell state) running through a factory. Things ride on it without being changed. Three workers stand next to it: one decides what to put *on* the belt (input gate), one decides what to throw *off* (forget gate), one decides what to *show* to the next department (output gate). Because the belt does not squash anything, an item put on at the beginning is still there at the end: long-term memory.

Board drill (write it by hand 3 times). $\mathbf{c}(t) = \mathbf{f} \odot \mathbf{c}(t-1) + \mathbf{i} \odot \mathbf{z}$ $\mathbf{h}(t) = \mathbf{o} \odot \tanh(\mathbf{c}(t))$ gates $= \sigma(W\mathbf{x}(t) + R\mathbf{h}(t-1) + \mathbf{b})$, cell input $= \tanh(\cdot)$ $\partial\mathbf{c}(t)/\partial\mathbf{c}(t-1) = \mathrm{diag}(\mathbf{f}(t)) \approx I$. Draw the cell: belt on top, three σ gates, one tanh in, one tanh out.

Q4.4 COURSE EXAM Explain the constant error carousel. Hochreiter

From: LSTM & Transformers (Hochreiter) · LSTM cell, constant error carousel · course exam

Say: In the original LSTM the cell connects to itself with weight 1 and a linear activation. So the local error in the cell is multiplied by exactly 1 at every step: it neither vanishes nor explodes. The gates protect this carousel: the input gate protects the cell from irrelevant inputs, the output gate protects other units from irrelevant cell content.

Write:

$\mathbf{c}(t) = \mathbf{c}(t-1) + \mathbf{i}(t) \odot \mathbf{z}(t) \;\Rightarrow\; \partial\mathbf{c}(t)/\partial\mathbf{c}(t-1) = I.$

Source: LSTM lecture WS24, CEC slides; Hochreiter & Schmidhuber (1997).

Q4.5 COURSE EXAM Why was the forget gate added? What about drift? Hochreiter

From: LSTM & Transformers (Hochreiter) · forget gate, drift, ticker steps · course exam

Say: Without a forget gate the cell can only accumulate; on long, continuous input streams the state grows without limit and the cell saturates. The forget gate lets the network reset its memory. Related: *internal state drift*, when inputs are mostly of one sign the state drifts away; the remedy in the original paper is to bias the input gates towards zero at the start (negative bias), so the cell writes only when needed.

Follow-up: "Ticker steps?" Letting the LSTM run extra steps after the input has ended, so it can process what it has stored before it must answer.

Q4.6 COURSE EXAM LSTM variants: peepholes, CIFG, GRU. Hochreiter

From: LSTM & Transformers (Hochreiter) · LSTM variants, GRU · course exam

Say: Peephole connections let the gates see the cell state $\mathbf{c}(t-1)$ directly. CIFG (coupled input and forget gate) sets $\mathbf{f} = 1 - \mathbf{i}$: write only as much as you forget. GRU (Cho et al. 2014) goes further: no separate cell, an update gate that acts as coupled input and forget gate, and a reset gate that decides how much the old state enters the candidate.

Write:

GRU: $\mathbf{u} = \sigma(\cdot)$, $\mathbf{r} = \sigma(\cdot)$, $\tilde{\mathbf{h}} = \tanh(W\mathbf{x}(t) + R(\mathbf{r} \odot \mathbf{h}(t-1)))$, $\mathbf{h}(t) = (1 - \mathbf{u}) \odot \mathbf{h}(t-1) + \mathbf{u} \odot \tilde{\mathbf{h}}$.

Follow-up: Other variants from the lecture: multidimensional LSTM (e.g. 2D images, several time axes), ConvLSTM (convolutions inside the gates, for video).

Q4.7 COURSE EXAM Dropout in LSTMs: why is it special? Hochreiter

From: LSTM & Transformers (Hochreiter) · LSTM dropout, zoneout · course exam

Say: Naive dropout on the recurrent state at every step multiplies survival probabilities over time, so a memory would survive with probability $(1-p)^T$: long-term memory is destroyed. So we use special forms: dropout only on the non-recurrent connections, variational dropout (the same mask at every time step), or zoneout (randomly *keep* the previous state instead of zeroing units). Dropout is switched off at evaluation.

Q4.8 ASKED BEFORE Bidirectional LSTM, and one label for a whole sequence. — Hochreiter, Schlüter

From: LSTM & Transformers (Hochreiter) · bidirectional RNN, sequence classification · asked in past defenses

Say: A bidirectional LSTM runs one LSTM forward and one backward and concatenates the two hidden states, so each step sees past and future; it needs the full sequence, so not for real-time use. For one label per sequence use the last hidden state or global average pooling over all states, then a classifier. Seq2seq: an encoder LSTM reads the input, a decoder LSTM writes the output.

4.3 Beyond LSTM: xLSTM, Mamba, Hopfield

2-minute summary ▪ Modern sequence models from Linz and elsewhere

xLSTM (Beck, Hochreiter et al. 2024): (1) *exponential gating* with a normalizer state, so the model can revise a storage decision (a new, more important input can overwrite the old one); (2) sLSTM: scalar memory with memory mixing and a stabilizer against overflow; (3) mLSTM: a *matrix* memory $C \in \mathbb{R}^{d\times d}$ updated by a covariance rule, fully parallelizable; its size does not grow with the sequence length.

$$\text{mLSTM: } C_t = f_t C_{t-1} + i_t\, \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{n}_t = f_t \mathbf{n}_{t-1} + i_t \mathbf{k}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \frac{C_t \mathbf{q}_t}{\max(|\mathbf{n}_t^\top \mathbf{q}_t|, 1)}, \quad i_t = \exp(\tilde{i}_t).$$

Mamba: a *selective* state space model, $\mathbf{h}_t = \bar{A}\mathbf{h}_{t-1} + \bar{B}\mathbf{x}_t$, $\mathbf{y}_t = C\mathbf{h}_t$, where B, C and the step Δ depend on the input; linear in sequence length. Modern Hopfield networks (Ramsauer, Hochreiter et al. 2020): the update $\boldsymbol{\xi}^{\text{new}} = X\,\text{softmax}(\beta X^\top \boldsymbol{\xi})$ is exactly Transformer attention.

Picture it. Old LSTM memory is a notebook where you write in pencil and can only fade old notes. xLSTM's exponential gate is a highlighter: a new, important note can outshine everything before it. The mLSTM matrix memory is a filing cabinet: store a value under a key ($\mathbf{v}\mathbf{k}^\top$), retrieve it later with a query ($C\mathbf{q}$).

Q4.9 COURSE EXAM What is new in xLSTM, and why? — Hochreiter

From: LSTM & Transformers (Hochreiter) · xLSTM (sLSTM, mLSTM) · course exam

Say: LSTM has three limits: it cannot revise a storage decision, its memory is a small scalar cell, and it cannot be parallelized over time because of memory mixing. xLSTM answers each: exponential gating with normalization revises storage decisions; mLSTM gives a matrix memory with key–value storage; and mLSTM drops memory mixing, so it trains in parallel like a Transformer while inference stays linear in the sequence length.

Write:

$C_t = f_t C_{t-1} + i_t \mathbf{v}_t \mathbf{k}_t^\top$, retrieval $C_t \mathbf{q}_t$; normalizer $\mathbf{n}_t = f_t \mathbf{n}_{t-1} + i_t \mathbf{k}_t$.

Source: LSTM lecture WS24, xLSTM exam questions.

Q4.10 PREDICTED What connects Hopfield networks and attention? — Hochreiter

From: IML research · modern Hopfield networks (Ramsauer et al. 2020) · predicted

Say: A modern Hopfield network stores patterns X and retrieves the one closest to a query $\boldsymbol{\xi}$ with $\boldsymbol{\xi}^{\text{new}} = X\,\text{softmax}(\beta X^\top \boldsymbol{\xi})$. With $\beta = 1/\sqrt{d_k}$ and learned projections this is exactly the attention formula. So attention is associative memory retrieval, and the network can store exponentially many patterns.

5 Attention and Transformers

2-minute summary ▪ Attention in two minutes

One idea. Instead of squeezing the past into one hidden state, let every token *look at every other token* and take a weighted mean. Each token makes three vectors by linear maps: a query (what I look for), a key (what I offer), a value (what I give). Weight = softmax of query · key.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \qquad \text{Attention}(Q,K,V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)V.$$

Shapes with N tokens: $X \in \mathbb{R}^{N\times d}$, $QK^\top \in \mathbb{R}^{N\times N}$ (rows sum to 1 after softmax), output $\in \mathbb{R}^{N\times d_v}$. Transformer block: multi-head self-attention → add & layer norm → position-wise MLP → add & layer norm. Key words: scaled dot product • multi-head • positional encoding • permutation equivariance • $O(N^2)$ • encoder/decoder, causal mask.

Picture it. A library. You come with a question (query). Every book has a title on its spine (key) and content inside (value). You compare your question with every title, trust the best matches most (softmax), and read a blend of their contents (weighted sum of values). Multi-head: several librarians, each searching by a different criterion.

Board drill (write it by hand 3 times). $\text{softmax}(QK^\top/\sqrt{d_k})V$; $A_{ij} = \dfrac{\exp(\mathbf{q}_i^\top \mathbf{k}_j/\sqrt{d_k})}{\sum_m \exp(\mathbf{q}_i^\top \mathbf{k}_m/\sqrt{d_k})}$; $\mathbf{o}_i = \sum_j A_{ij}\mathbf{v}_j$; $\text{MultiHead} = \text{Concat}(\text{head}_1,\dots,\text{head}_h)W_O$.

Q5.1 ASKED BEFORE Explain the Transformer architecture. — Rekabsaz, Lehner

From: LSTM & Transformers (Hochreiter) · Transformer; NLP course · asked in past defenses

Say: The input tokens are embedded and get a positional encoding. The encoder is a stack of blocks, each with multi-head self-attention and a feed-forward network, both with residual connections and layer normalization. The decoder has the same blocks, but its self-attention is *masked* (a token cannot see the future), and it has a cross-attention layer whose keys and values come from the encoder. BERT uses only the encoder (masked language modelling); GPT only the decoder.

Follow-up: "Why residual + layer norm?" Stable gradients through deep stacks, like ResNet.

Q5.2 ASKED BEFORE Write the matrix multiplications of self-attention. — Klambauer

From: LSTM & Transformers (Hochreiter) · scaled dot-product attention · asked in past defenses

Say: Three projections of the same input, one $N \times N$ score matrix, a row-wise softmax, and a product with the values.

Write:

$X \in \mathbb{R}^{N\times d}$; $Q, K \in \mathbb{R}^{N\times d_k}$, $V \in \mathbb{R}^{N\times d_v}$; $S = QK^\top/\sqrt{d_k} \in \mathbb{R}^{N\times N}$; $A = \text{softmax}_{\text{row}}(S)$; $O = AV \in \mathbb{R}^{N\times d_v}$.

Follow-up: "Why divide by $\sqrt{d_k}$?" With random unit-variance entries the dot product has variance d_k; dividing keeps it at 1, so the softmax does not saturate and gradients stay alive.

Q5.3 ASKED BEFORE Permutation invariance: why a problem, how to fix? — Lehner

From: ML: Supervised Techniques (Lehner) unit 7; LSTM & Transformers (Hochreiter) · asked in past defenses

Say: Self-attention treats the tokens as a set: shuffle the input and the output is shuffled the same way (permutation equivariant). Word order is lost. Fix: add a positional encoding to each token, fixed sinusoids $\sin(p/10000^{2i/d})$, $\cos(\cdot)$, learned position vectors, or relative encodings such as RoPE.

Q5.4 ASKED BEFORE RNN versus Transformer: complexity and trade-offs. Lehner

From: ML: Supervised Techniques (Lehner) unit 7; LSTM & Transformers (Hochreiter) · asked in past defenses

Say: RNN: $O(N)$ in sequence length, constant memory for inference, but sequential, so slow to train and long paths between distant tokens. Transformer: $O(N^2)$ time and memory, but fully parallel in training and every pair of tokens is one step apart. xLSTM and Mamba try to get both: parallel training and linear inference.

Write:

RNN $O(N\,d^2)$ Transformer $O(N^2 d)$.

6 Classical ML asked in the same slot

2-minute summary ▪ SVM, kernels, VC, likelihood, REINFORCE

SVM: largest-margin separating hyperplane; soft margin with slack ξ_n and penalty C. Kernel trick: the dual needs only dot products, so replace them by $k(\mathbf{x}, \mathbf{x}')$. VC dimension: the largest point set a model class can shatter; linear in $\mathbb{R}^d$: $d+1$. MLE: choose the parameters that make the data most likely; overfits with little data. REINFORCE: policy gradient by the log-derivative trick.

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_n \xi_n \;\text{ s.t. }\; y_n(\mathbf{w}^\top \mathbf{x}_n + b) \geq 1-\xi_n,\; \xi_n \geq 0; \qquad \nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t\Big].$$

Picture it. SVM: build the widest possible road between two villages; the houses standing right at the road's edge are the support vectors, they alone decide where the road goes. Kernel: lift the map into 3D, where a curved border becomes a flat wall, without ever drawing the 3D map.

Board drill (write it by hand 3 times). Primal SVM (above) • dual $\max_{\boldsymbol{\alpha}} \sum \alpha_n - \frac{1}{2}\sum_{n,m} \alpha_n\alpha_m y_n y_m k(\mathbf{x}_n,\mathbf{x}_m)$, $0 \leq \alpha_n \leq C$, $\sum \alpha_n y_n = 0$ • RBF $k = \exp(-\|\mathbf{x}-\mathbf{x}'\|^2/2\sigma^2)$ • $\mathcal{L}(\theta) = \prod_n p(\mathbf{x}_n;\theta)$, $\hat{\theta} = \arg\max \sum_n \log p(\mathbf{x}_n;\theta)$ • REINFORCE loss $-\sum_t \log \pi_\theta(a_t|s_t)(G_t - b)$.

Q6.1 ASKED BEFORE Explain the SVM and write the optimization problem with its constraints. Kofler, Hochreiter

From: ML: Supervised Techniques · SVM · asked in past defenses

Say: The SVM separates two classes with the hyperplane of maximal margin $2/\|\mathbf{w}\|$. Maximizing the margin is minimizing $\frac{1}{2}\|\mathbf{w}\|^2$. The soft margin allows violations with slack variables, paid for with C. Only points on or inside the margin get $\alpha_n > 0$: the support vectors.

Write:

(primal and dual as in the board drill); decision $f(\mathbf{x}) = \text{sign}\big(\sum_n \alpha_n y_n k(\mathbf{x}_n, \mathbf{x}) + b\big)$.

Follow-up: "Large C?" Few violations, narrow margin, risk of overfitting. "Small C?" Wide margin, more violations.

Q6.2 ASKED BEFORE What are kernels, why important, what is the kernel trick? Kofler

From: ML: Supervised Techniques · kernels · asked in past defenses

Say: A kernel is a dot product in a feature space, $k(\mathbf{x},\mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}')\rangle$. Many algorithms (SVM dual, kernel ridge regression, kernel PCA) use data only through dot products, so we plug in the kernel and get a non-linear method at linear cost, without computing ϕ, which may even be infinite-dimensional (RBF). A valid kernel gives a positive semi-definite Gram matrix (Mercer).

Q6.3 ASKED BEFORE VC dimension: what is it, what is it used for? — Kofler

From: ML: Supervised Techniques; statistical learning theory · asked in past defenses

Say: The largest number of points that the model class can shatter, i.e. classify correctly under every possible labelling. Lines in 2D: 3. It measures capacity and appears in generalization bounds: with probability $1-\eta$, $R_{\text{gen}} \le R_{\text{emp}} + \sqrt{\frac{h(\log(2N/h)+1)-\log(\eta/4)}{N}}$, with h the VC dimension. Small h or large $N \to$ small gap.

Q6.4 ASKED BEFORE Likelihood, maximum likelihood, expectation maximization. — Widmer, Kofler

From: Probabilistic Models (Widmer); ML: Unsupervised Techniques · asked in past defenses

Say: The likelihood is the probability of the observed data as a function of the parameters. MLE maximizes it (in practice the log). Problem: with little data it overfits, e.g. the sunrise problem, where an unseen event gets probability zero; a prior (MAP, Laplace smoothing) fixes it. EM is used when there are hidden variables (mixture models, HMMs): the E-step computes the posterior of the hidden variables with the current parameters, the M-step maximizes the expected complete-data log-likelihood; the likelihood never decreases.

Write:

$\hat{\theta}_{\text{MLE}} = \arg\max_\theta \sum_n \log p(\mathbf{x}_n;\theta)$; link to ERM: squared loss $\Leftrightarrow$ Gaussian MLE, cross-entropy $\Leftrightarrow$ categorical MLE.

Q6.5 ASKED BEFORE Derive the REINFORCE loss. — Hochreiter

From: Deep Reinforcement Learning · policy gradient · asked in past defenses

Say: We maximize the expected return over trajectories. The gradient of an expectation is turned into an expectation of a gradient with the log-derivative trick, $\nabla p = p\nabla \log p$. The environment dynamics do not depend on θ, so only the policy terms remain.

Write:

$\nabla_\theta J = \nabla_\theta \int p_\theta(\tau) G(\tau)\,\mathrm{d}\tau = \int p_\theta(\tau)\nabla_\theta \log p_\theta(\tau) G(\tau)\,\mathrm{d}\tau$, $\log p_\theta(\tau) = \log p(s_0) + \sum_t \big[\log \pi_\theta(a_t|s_t) + \log p(s_{t+1}|s_t,a_t)\big] \Rightarrow \nabla_\theta J = \mathbb{E}_\tau\big[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t\big]$.

Follow-up: "Why a baseline b?" Subtracting $b(s_t)$ does not change the expected gradient but lowers its variance.

> Picture it. Training a dog: after a good walk (high return), make every action of that walk a bit more likely; after a bad one, a bit less.

Q6.6 ASKED BEFORE Confusion matrix, precision, recall, AUC. — general

From: ML: Supervised Techniques; AI and Visualization · evaluation · asked in past defenses

Say: Precision $= TP/(TP+FP)$: of what I called positive, how much is right. Recall $= TP/(TP+FN)$: of all positives, how many I found. The ROC curve plots true-positive rate against false-positive rate over all thresholds; AUC is the probability that a random positive scores higher than a random negative (0.5 chance, 1 perfect).

7 rCFD and CFD (Prof. Pirker)

2-minute summary ▪ rCFD in two minutes

One idea. Solve the expensive flow once, record *how the fluid moves* as cell-to-cell maps, then replay those maps many times to carry a scalar (species, temperature). Momentum and pressure are never solved again: hundreds of times faster.

$$T_j^* = \sum_i M_{ji}^{(f)} T_i, \qquad \mathbf{T}^{n+1} = \mathcal{B}\Big(\mathcal{C}\big(\mathcal{S}(\mathbf{M}^{(f_n)}\mathbf{T}^n)\big)\Big), \qquad R_{fg} = \|\Phi^{(f)} - \Phi^{(g)}\|.$$

map $\mathbf{M}$ (advection) • face swap $\mathcal{S}$ (diffusion) • correction $\mathcal{C}$ (your work) • conservation constraint $\mathcal{B}$. Pirker's three bold failures: (1) mass imbalance → global monitor + mass-acting face swaps; (2) unphysical diffusion → locally adapted swaps (sub-grid viscosity); (3) weak recurrence prominence → recurrence islands. Works: recurring flow, passive scalar. Fails: the scalar drives the flow, or the process changes state → multi-state rCFD.

Picture it. A GPS that recorded one drive through the city. Next time you only replay the recorded route (cheap), you do not recompute it. It works as long as the streets are the same. If a street is closed (a new boundary condition, like the cooled lid), the recording does not know the detour: you need a new recording for that situation.

Board drill (write it by hand 3 times). $\mathbf{T}^{n+1} = \mathcal{B}(\mathcal{C}(\mathcal{S}(\mathbf{M}\mathbf{T}^n)))$ • $\Delta_{\text{swap}} = f_{\text{sd}} V\,(T_{c_1} - T_{c_0})/2$, $D_{\text{eq}} = f_{\text{sd}} c_s^2/(2\Delta t)$ • limiter $\Delta q_f = s\min(Q_h^+, Q_c^-)$ • $Co = u\Delta t/\Delta x < 1$, $d = \Gamma\Delta t/\Delta x^2 < 0.5$ • $Re = uL/\nu$.

Q7.1 PREDICTED Flow-based versus transport-based rCFD? Pirker

From: Numerical Methods in Fluid Mechanics (Pirker) · rCFD lecture, slides 16–34 · predicted

Say: Flow-based: stitch recorded flow sequences into a long artificial flow and solve a transport equation on it (Lichtenegger & Pirker 2016, about 100× faster). Transport-based: replace convection by cell-to-cell shifts and diffusion by face swaps, no equation solved (Pirker & Lichtenegger 2018, steel ladle 26 h → 1.6 s). My thesis uses the transport-based form.

Q7.2 PREDICTED Recurrence matrix and prominence – and your case? Pirker

From: Numerical Methods in Fluid Mechanics (Pirker) · rCFD slides 50–51; thesis Sec. 2.4 · predicted

Say: The recurrence norm compares two recorded states; all pairs form the matrix. A row (recurrence vector) rises and oscillates about a plateau; deep dips = the flow comes back = we can jump. Prominence $P_{\text{rel}} = \langle d_{\max} - d_{\min}\rangle/\bar{d}$, required 0.05–0.1. Mine: median 0.028, two blocks (filling, settling), no return → sequential replay.

Q7.3 PREDICTED Why did the recorded local diffusivity not help? Pirker

From: Thesis Sec. 2.8.1; Numerical Methods in Fluid Mechanics (Pirker) · explicit limits · predicted

Say: The exchange is explicit and capped at half a cell per face and step, a diffusion number $\text{Di} = D\Delta t/c_s^2 \le 0.5$. The recorded D asks for $\text{Di} \approx 4$ everywhere, so every face hits the cap and the field behaves like $f_{\text{sd}} = 0.5$ (three replays bit-identical). An implicit exchange or a smaller step would let it act. This is the explicit stability limit of the lecture, $d < 0.5$.

Q7.4 PREDICTED Numerical diffusion: where in your thesis? Pirker

From: Numerical Methods in Fluid Mechanics (Pirker) · FV II numerical diffusion; thesis Sec. 4.2 · predicted

Say: First-order upwind is stable but adds false diffusion that smears fronts. In my replay the smearing comes from the face exchange, not the maps (maps only keep the front at 2–4 mm). My correction is an anti-diffusive step bounded like flux-corrected transport: conservative, no new extremum, front 15.7 → 5.3/4.5 mm against 5.0 mm.

Q7.5 PREDICTED rCFD and digital twins. Pirker

From: Numerical Methods in Fluid Mechanics (Pirker) · rCFD slides 59–72 digital twins · predicted

Say: CSTR models are instant but blind to heterogeneity; CFD sees it but takes days. rCFD built on CFD can run alongside the process as a soft sensor ($T_{\max}$, σ_T), aligned with live data to stop error accumulation. My thesis adds two warnings: a boundary condition can switch the flow regime, and the heat budget must use the same fluid properties as the recording.

Q7.6 PREDICTED CFD basics he may check quickly. Pirker

From: Numerical Methods in Fluid Mechanics (Pirker) · GovEq, FV, SIMPLE, turbulence · predicted

Say: Re $= uL/\nu$ (mine 99, laminar). Material derivative $\mathrm{D}/\mathrm{D}t = \partial_t + \mathbf{u}\cdot\nabla$. FVM: integrate over cells, fluxes through faces, locally conservative, $a_P c_P = \sum a_{NB} c_{NB} + b$. SIMPLE: guess p, solve $\mathbf{u}^*$, pressure correction from continuity, correct, repeat. Turbulence: cascade to Kolmogorov $\eta = (\nu^3/\varepsilon)^{1/4}$; RANS models all, LES the small eddies, DNS none; $\nu_t = C_\mu k^2/\varepsilon$. Boussinesq: constant density except in the gravity term (mine: 1.5 per cent density change).

8 Predicted questions and rapid-fire drill

8.1 Thesis questions (first 5 minutes)

2-minute summary ▪ Your thesis in two minutes

rCFD records a flow once and replays it to carry a scalar. It was built for passive scalars; I tested it on temperature, which drives its own flow (buoyant jet in a $75 \times 75 \times 15$ mm tank, Re 99). (1) No single mixing constant is best: the outlet sits under the front, so smearing drains heat. (2) A bounded, vertically weighted anti-diffusive correction brings the front from 15.7 to 5.3/4.5 mm (reference 5.0), no value outside the data range. (3) Under wall cooling the replay was 4.34 K too warm because the reference used c_p of air (1006.43) and the replay of water (4182); on one fluid 0.66 K, corrected 0.06 K on 9.62 K. (4) What remains is missing transport: a recorded flux cuts the heterogeneity error from 1.66 to 0.26 K. Take-home: a database belongs to a flow regime.

Picture it. A cooking video recorded once (preparation) and replayed to cook many times (replay). It works for soup that just gets stirred (passive scalar). For bread that rises by itself (buoyancy), the video does not show how your dough will move, so you must correct it.

Q8.1 ASKED BEFORE What problem did you solve, and why ML/AI? chair

From: Thesis, Appendix A1 (story summary) · asked in past defenses

Say: I measured what a recurrence replay loses when the scalar is not passive, and repaired most of it without solving the flow again. It is AI for simulation: expensive solver data is reused by a cheap model; choosing the next frame is a nearest-neighbour search; and my main result is a generalization result: the database covers a regime, and a new boundary condition is out-of-distribution.

Q8.2 ASKED BEFORE What would you do differently? chair

From: Thesis, Ch. 8 and future work · asked in past defenses

Say: Check the material properties of both simulations on the first day: the specific-heat mismatch cost the most time. Then an implicit exchange so the recorded diffusivity can act.

Q8.3 PREDICTED How would you solve your problem with deep learning? Klambauer

From: Thesis × DL: graph neural networks · predicted

Say: Learn the correction $\mathcal{C}$ with a graph neural network on the mesh: input the field and the recorded map, output the next field, loss = distance to the reference plus a penalty on the energy balance. The risk is the same as in my thesis: it only generalizes inside the regime of its training data.

Q8.4 PREDICTED Could an LSTM replace the replay? Hochreiter

From: Thesis × LSTM & Transformers (Hochreiter) · predicted

Say: An LSTM (or xLSTM) could learn the time evolution of a low-dimensional state, e.g. the POD coefficients of Figure 5.9, as a sequence-to-sequence model. It would be cheap and could carry memory of the filling phase, which is exactly where my replay lags. But it needs many trajectories to generalize, and without a conservation constraint it can drift in energy, which rCFD guarantees by construction.

8.2 Rapid-fire (say it in one breath)

Question	Answer
Most powerful learning machine?	The brain; closest method: neural networks.
Delta, softmax + CE?	$\hat{\mathbf{y}} - \mathbf{y}$.
Activation from Linz?	SELU (self-normalizing), ELU.
Who found vanishing gradients?	Hochreiter, 1991, Linz.
Constant error carousel?	Cell self-loop weight 1, linear: error flows unchanged.
Forget gate added by?	Gers, Schmidhuber, Cummins, 2000.
GRU gates?	Update (coupled input/forget) and reset.
BPTT memory?	Linear in T; RTRL: independent of T, $O(N^4)$ per step.
Truncated BPTT?	Cuts the past after k steps; approximate gradient.
Gradient clipping fixes?	Exploding only, not vanishing.
Why $\sqrt{d_k}$?	Keeps dot-product variance at 1; softmax does not saturate.
Attention complexity?	$O(N^2)$; RNN $O(N)$.
Positional encoding why?	Attention is permutation-equivariant.
xLSTM two ideas?	Exponential gating; matrix memory (mLSTM).
Hopfield ↔ attention?	$X\,\mathrm{softmax}(\beta X^\top \boldsymbol{\xi}) =$ attention.
UAT hidden layers?	One, finitely many units.
Non-differentiable loss?	0–1 loss; use cross-entropy or hinge.
Regularization?	Weight decay, dropout, early stopping, augmentation, FMS.
Optimizers?	SGD, momentum, AdaGrad, RMSprop, Adam.
Output activation, regression?	Linear + squared loss.
VC dim. of lines in 2D?	3.
Kernel trick?	Dot products → $k(\mathbf{x}, \mathbf{x}')$.
MLE problem?	Overfits with little data (sunrise problem).
Your Re / grid / frames?	99 / 1 mm cubes / 600 frames.
Your speed-up?	About 260 times.

9 Conversation chains

Past students all say it: the exam is a conversation. The examiner takes a word from your answer and asks about it. So every word you say is a door. This chapter turns that into a tool: each answer ends with a *hook*, one sentence that names the next topic. You open the door you want, to a room you have prepared.

2-minute summary ▪ How to use the chains

Rule 1. Answer in 2–3 sentences, then say the hook. Stop. Let the examiner walk through the door. Rule 2. Only say words you can explain. "Batch norm" in your answer means a batch-norm question in two seconds. Rule 3. If the examiner goes elsewhere, fine: jump to the matching step of another chain. Rule 4. Learn a chain as a *story*, not as a list: each step is the answer to the problem of the step before. Symbols: Q question • A your short answer • → hook the sentence that opens the next door.

Chain 1 – From data to batch normalization (Klambauer, Hochreiter)

Picture it. A story in one line: *we want small risk → we need gradients → gradients vanish → we fix the signal size → with activations, with initialization, with normalization.*

Q. What is the goal of learning?
A. Small error on new data, the generalization error. We cannot measure it, so we minimize the empirical risk $R_{\text{emp}} = \frac{1}{N}\sum_n L(y_n, g(\mathbf{x}_n; \mathbf{w}))$ on the training data.
→ hook: *To minimize it we need a differentiable loss, so we use cross-entropy instead of the 0–1 loss.*

Q. Why cross-entropy?
A. It is differentiable, and minimizing it is maximum likelihood for a categorical output. With softmax at the output the delta becomes simply $\hat{\mathbf{y}} - \mathbf{y}$.
→ hook: *We then minimize it with gradient descent, in practice with mini-batches.*

Q. Gradient descent and mini-batches?
A. $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla R_{\text{emp}}$. A mini-batch gives a good, cheap gradient estimate, runs in parallel on a GPU, and its noise helps to find flat minima.
→ hook: *The gradient itself we get with backpropagation.*

Q. Backpropagation?
A. Forward pass, store s and a. Delta at the output, then down: $\delta_j = f'(s_j)\sum_i \delta_i w_{ij}$. Gradient $\partial L/\partial w_{ij} = \delta_i a_j$.
→ hook: *Because the delta is multiplied by f' and W in every layer, it can vanish.*

Q. Vanishing gradients?
A. A product of many Jacobians shrinks or grows exponentially. With sigmoid $|f'| \leq 0.25$. Found by Hochreiter in 1991.
→ hook: *Three fixes: the activation function, the initialization, and normalization.*

Q. Which activation helps?
A. ReLU does not saturate for positive inputs. SELU, from Linz, even keeps mean 0 and variance 1 by itself: self-normalizing, with a stable fixed point.
→ hook: *The same goal, a constant variance, is also the idea of good initialization.*

Q. Initialization?
A. Random, to break symmetry, with variance $1/n_{\text{in}}$ (or $2/n_{\text{in}}$ for ReLU) so the signal keeps its size forward and backward.
→ hook: *During training the statistics drift again, and batch normalization corrects that.*

Q. Batch normalization?
A. Standardize each pre-activation over the mini-batch, $\hat{s} = (s - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$, then learn scale and shift, $\gamma\hat{s} + \beta$. Training is faster and more stable, and it regularizes a little. Layer norm does the same over the features instead of the batch, which suits RNNs and Transformers.
→ hook: *SELU reaches the same effect without any batch statistics, which is why it was developed here.*

Chain 2 – From RNN to xLSTM (Hochreiter)

Picture it. *RNNs forget → LSTM remembers on a conveyor belt → Transformers look at everything, but pay N^2 → xLSTM brings memory back at linear cost.*

Q. What is an RNN?
A. A network with a hidden state that is fed back: $\mathbf{s}(t) = W\mathbf{x}(t) + R\mathbf{a}(t-1)$. Same weights at every step, any sequence length, Turing complete.
→ hook: *We train it with backpropagation through time.*

Q. BPTT?

A. Unroll in time and backprop: $\boldsymbol{\delta}(t) = \mathrm{diag}(f')(V^\top \partial L(t)/\partial \hat{\mathbf{y}} + R^\top \boldsymbol{\delta}(t+1))$. Memory grows with the sequence length; RTRL avoids that but costs $O(N^4)$ per step.
→ hook: *The problem is that the error from far in the past vanishes.*

Q. Why does it vanish?
A. At every step it is multiplied by $R^\top \mathrm{diag}(f')$; after k steps by the k-th power. Clipping helps against exploding, not vanishing.
→ hook: *LSTM solves it with the constant error carousel.*

Q. The constant error carousel?
A. A memory cell updated by addition, $\mathbf{c}(t) = \mathbf{f} \odot \mathbf{c}(t-1) + \mathbf{i} \odot \mathbf{z}$, so $\partial \mathbf{c}(t)/\partial \mathbf{c}(t-1) = \mathrm{diag}(\mathbf{f}) \approx I$. The error rides the belt unchanged. Gates decide what to write, keep and read.
→ hook: *The forget gate was added later, and GRU simplified the gates.*

Q. Forget gate and GRU?
A. Gers et al. 2000: the forget gate lets the cell reset on long streams. GRU couples input and forget into one update gate and adds a reset gate; fewer parameters, similar results.
→ hook: *LSTMs still read step by step; attention removed that bottleneck.*

Q. Attention and Transformers?
A. $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$: every token looks at every token in one step, fully parallel. The price is $O(N^2)$ in the sequence length.
→ hook: *xLSTM tries to keep the parallel training but with linear cost.*

Q. xLSTM?
A. Exponential gating with a normalizer, so storage decisions can be revised; and mLSTM, a matrix memory $C_t = f_t C_{t-1} + i_t \mathbf{v}_t \mathbf{k}_t^\top$, parallel in training, linear in inference.
→ hook: *And the attention formula itself is the update rule of a modern Hopfield network.*

Chain 3 – Generalization and regularization (Klambauer)

Picture it. *Fit the data → but not too well → measure it honestly → control it.*

Q. What is overfitting?
A. Training error low, test error high: the model learned the noise. Too much capacity for the data.
→ hook: *Capacity can be measured, for example by the VC dimension.*

Q. VC dimension?
A. The largest point set the model class can shatter; lines in 2D: 3. Bounds say the gap between test and training error grows with it and shrinks with N.
→ hook: *In practice we do not use the bound, we estimate the test error directly.*

Q. How?
A. Held-out test set, or L-fold cross-validation, which is almost unbiased and uses all data.
→ hook: *If the estimate shows overfitting, we regularize.*

Q. Regularization?
A. Weight decay $\frac{\lambda}{2}\|\mathbf{w}\|^2$, dropout, early stopping, data augmentation, Flat Minimum Search. They all limit effective capacity.
→ hook: *Interestingly, very large networks can generalize even without it: double descent.*

Q. Double descent?
A. Past the interpolation point the test error falls again as the model grows; the classical U-curve is only the first half.

Chain 4 – From your thesis into ML (Klambauer, chair)

Picture it. *rCFD is data reuse → data reuse means generalization → generalization fails outside the training regime → a learned model would face the same limit.*

Q. Why is your thesis AI?
A. The expensive solver produces data once, and a cheap model reuses it. Choosing the next frame is a nearest-neighbour search over recorded states.
→ hook: *So the key question is the same as in ML: how far does it generalize?*

Q. How far does it generalize?
A. Within its regime. The adiabatic recording fails under the cooled lid because cold plumes create a motion it never saw: out-of-distribution.
→ hook: *A learned surrogate would have the same limit, but it could learn the missing correction.*

Q. Which model would you use?
A. A graph neural network on the mesh: cells are nodes, faces are edges, message passing like the face exchange. Loss: distance to the reference plus an energy-balance penalty.
→ hook: *The penalty matters, because conservation is what rCFD guarantees by construction.*

Q. Why conservation?
A. My corrections are conservative: every face exchange appears twice with opposite signs, so the heat in the tank only moves. A learned model without that constraint can drift in energy.

Chain 5 – From numerical diffusion to your limiter (Pirker)

Picture it. *Schemes smear → explicit steps have limits → my cap is such a limit → unsmearing needs a limiter.*

Q. What is numerical diffusion?
A. First-order upwind is stable but adds false diffusion that smears fronts, error $\propto \Delta x$. Central differencing is sharper but oscillates when the grid Péclet number exceeds 2.
→ hook: *In my replay the smearing comes from the face exchange, which is an explicit step.*

Q. Limits of explicit steps?
A. Convection needs $Co = u\Delta t/\Delta x < 1$, diffusion $d = \Gamma\Delta t/\Delta x^2 < 0.5$.
→ hook: *This is exactly why the recorded diffusivity could not act.*

Q. Why not?
A. The exchange is capped at $\mathrm{Di} \leq 0.5$; the recorded field asked for about 4. Every face hit the cap, so it behaved like the constant $f_{\mathrm{sd}} = 0.5$.
→ hook: *To undo the smearing I needed anti-diffusion, and that needs a limiter.*

Q. Your limiter?
A. Negative diffusion is ill-posed and blows up at the grid scale. So I move heat up the gradient only as far as both cells' local bounds allow, $\Delta q_f = s\min(Q_h^+, Q_c^-)$, like flux-corrected transport: conservative, no new extremum.
→ hook: *And I weight it towards the vertical, because the front is horizontal.*

Doors to open and doors to keep closed

Say it (you have a room behind it)	Avoid unless asked (deep rooms)
empirical risk, cross-entropy, mini-batch	natural gradient, Fisher information
delta error, vanishing gradient, SELU	neural tangent kernel
constant error carousel, forget gate, GRU	multidimensional LSTM details
attention, $\sqrt{d_k}$, positional encoding	rotary embeddings in depth
flux-corrected transport, limiter, conservative	Harten's TVD theory
out-of-distribution, regime, nearest neighbour	Koopman operators, propagators in depth

10 Hot topics and the Linz school

Examiners like to end with "what is new in the field?" or "what do you know about our work?". This chapter gives you short, safe answers. It covers the Institute for Machine Learning (IML) at JKU first, because Prof. Hochreiter and Prof. Klambauer will enjoy talking about their own work, and then the general hot topics of 2025–2026.

10.1 The Linz school (IML, JKU)

2-minute summary ▪ Linz in two minutes: one timeline

1991	Vanishing gradient analysed (Hochreiter, diploma thesis)
1997	LSTM (Hochreiter & Schmidhuber); Flat Minimum Search
2014	DeepTox wins the Tox21 challenge (Mayr, Klambauer, Unterthiner, Hochreiter)
2015/17	ELU (Clevert et al.) and SELU, self-normalizing networks (Klambauer et al.)
2017	FID, the standard score for generative image models (Heusel et al.)
2018/19	Fréchet ChemNet Distance (Preuer et al.); RUDDER, reward redistribution (Arjona-Medina et al.)
2020	Hopfield Networks is All You Need (Ramsauer et al.): attention = Hopfield update
2024	xLSTM (Beck et al.); Vision-LSTM; NeuralDEM with Prof. Pirker's group
2025	xLSTM 7B language model and TiRex time-series model (NXAI, Hochreiter's company)

The common thread: memory and gradient flow (LSTM, xLSTM, Hopfield), stable deep training (SELU, flat minima), and AI for science (DeepTox, molecules, physics simulation).

Picture it. Linz is the "memory school" of deep learning. Its big ideas are all about keeping information alive: the error in time (LSTM), the signal in depth (SELU), patterns in memory (Hopfield), and now long sequences at linear cost (xLSTM).

Q10.1 PREDICTED What do you know about the research of our institute? Hochreiter, Klambauer

From: IML research overview · predicted

Say: The institute is known for LSTM and the analysis of vanishing gradients, for self-normalizing networks with SELU, and for modern Hopfield networks, which showed that attention is an associative memory. Recently xLSTM brought LSTM back at scale, with exponential gating and a matrix memory, and the company NXAI built a 7-billion-parameter xLSTM language model and the TiRex forecasting model. There is also strong work on AI for science: drug discovery since DeepTox, and physics simulation.

→ hook: *For me the physics side is the closest: NeuralDEM was done together with my supervisor's group.*

Q10.2 PREDICTED What is NeuralDEM, and how does it relate to your thesis? — Pirker, Klambauer

From: NeuralDEM (Alkin, Brandstetter, Pirker et al. 2024) · predicted

Say: NeuralDEM (Alkin, Brandstetter, Pirker, Lichtenegger and others, 2024) replaces the discrete element method for industrial particulate flows with a deep-learning surrogate that runs in real time. It treats the particles as a continuous field and learns their evolution with a transformer-type model. It is the learned cousin of rCFD: both reuse expensive simulation data to run fast, rCFD by storing operators, NeuralDEM by learning them.

Follow-up: "Which is better?" rCFD conserves the scalar by construction and needs no training; a learned model can generalize to new states if it has seen enough of them. My thesis shows the limit they share: a new flow regime is out of the data.

Q10.3 PREDICTED Why would anyone use an LSTM today, when Transformers exist? — Hochreiter

From: xLSTM (Beck et al. 2024); LSTM & Transformers (Hochreiter) · predicted

Say: Transformers cost $O(N^2)$ in the context length and their memory grows with it at inference. xLSTM keeps a fixed-size state, so inference is linear in length and constant in memory, which matters for long contexts, robotics and edge devices. With exponential gating and the mLSTM matrix memory it reaches Transformer-level language modelling, and mLSTM trains in parallel.

Write:

$$C_t = f_t C_{t-1} + i_t \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{h}_t = \mathbf{o}_t \odot \frac{C_t \mathbf{q}_t}{\max(|\mathbf{n}_t^\top \mathbf{q}_t|, 1)}.$$

→ hook: *The same idea works for images (Vision-LSTM) and for time series (TiRex).*

Q10.4 PREDICTED What is RUDDER? — Hochreiter

From: RUDDER (Arjona-Medina et al. 2019); Deep RL · predicted

Say: In reinforcement learning with delayed rewards, credit assignment is slow. RUDDER trains an LSTM to predict the return of an episode and then redistributes the reward to the steps where the prediction jumps, i.e. to the actions that actually mattered. The expected future reward becomes close to zero, so learning is much faster.

Picture it. A football goal is credited not only to the scorer, but to the pass that made the difference.

Q10.5 PREDICTED What is the FID? — Klambauer

From: FID (Heusel et al. 2017) · predicted

Say: The Fréchet Inception Distance measures how close generated images are to real ones: fit a Gaussian to Inception-network features of both sets and compute the Fréchet distance.

Write:

$$\mathrm{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \mathrm{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}).$$

Follow-up: The same idea for molecules is the Fréchet ChemNet Distance.

10.2 Hot topics 2025–2026

2-minute summary ▪ Six trends to name

1. Reasoning LLMs: models trained with reinforcement learning on verifiable rewards (maths, code) to think longer before answering: test-time compute.
2. Alignment: RLHF and its simpler successor DPO learn from human preferences.
3. Efficient sequence models: xLSTM, Mamba (state space), linear attention; and mixture-of-experts Transformers that activate only a few experts per token.
4. Generative models: diffusion and flow matching for images, video, molecules.

5. Agents and retrieval: LLMs that call tools and read documents (RAG).
6. AI for science: protein structure (AlphaFold 3), weather (GraphCast, GenCast), neural surrogates for CFD and particle flows (NeuralDEM), foundation models for physics.

Plus regulation: the EU AI Act (risk-based rules), which links to your AI and Law courses.

Picture it. The field is moving from "bigger models" to "smarter use of compute": think longer (reasoning), use only the needed part (mixture of experts), remember cheaply (xLSTM, Mamba), and ask tools instead of guessing (agents).

Q10.6 PREDICTED What is flow matching? (You had it in ML: Advanced Techniques.) general

From: ML: Advanced Techniques · flow matching unit · predicted

Say: A generative model that learns a velocity field which moves noise to data along a path. With the straight path $\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1$ the target velocity is simply $\mathbf{x}_1 - \mathbf{x}_0$, so training is a plain regression. Sampling solves the ODE $\mathrm{d}\mathbf{x}/\mathrm{d}t = \mathbf{v}_\theta(\mathbf{x}, t)$.

Write:

$L(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\mathbf{x}_1} \left\| \mathbf{v}_\theta(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0) \right\|^2$.

→ hook: *Diffusion models are the same family with a noisy, curved path.*

Q10.7 PREDICTED Diffusion models in two sentences? general

From: ML: Advanced Techniques · diffusion (VDM) · predicted

Say: Add Gaussian noise step by step until the data is pure noise, and train a network to predict the added noise; sampling runs the process backwards. Loss: $\mathbb{E}\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t)\|^2$.

Q10.8 PREDICTED How are LLMs aligned? RLHF versus DPO. general

From: Current literature: RLHF, DPO · predicted

Say: RLHF: train a reward model on human preference pairs, then optimize the LLM against it with RL (PPO), with a KL penalty to stay close to the original model. DPO skips the reward model and optimizes the preferences directly with a classification-like loss.

Write:

DPO: $L = -\log \sigma\Big(\beta \log \dfrac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \dfrac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\Big)$.

→ hook: *Reasoning models go one step further: they learn from rewards that can be checked automatically.*

Q10.9 PREDICTED What are reasoning models and test-time compute? general

From: Current literature: reasoning models · predicted

Say: Models that write out intermediate steps (chain of thought) before the answer, trained with reinforcement learning on tasks whose answers can be verified, like maths or code. Spending more computation at inference, by thinking longer or sampling several solutions, improves accuracy: test-time compute becomes a new scaling axis next to model size and data.

Q10.10 PREDICTED Mixture of experts and state space models? general

From: Current literature: MoE, Mamba · predicted

Say: Mixture of experts: many feed-forward "experts", a router sends each token to the top-k, so the model has many parameters but uses few per token: $\mathbf{y} = \sum_{i \in \text{top-}k} g_i(\mathbf{x}) E_i(\mathbf{x})$. State space models like Mamba: a linear recurrence $\mathbf{h}_t = \bar{A}\mathbf{h}_{t-1} + \bar{B}\mathbf{x}_t$ with input-dependent (selective) parameters; linear in sequence length, like xLSTM.

Q10.11 PREDICTED Where is AI for simulation going? Pirker, general

From: AI for science literature; NeuralDEM · predicted

Say: From single surrogates to foundation models for physics: neural operators and transformer surrogates trained on many simulations, weather models that beat numerical forecasts in speed, and industrial real-time twins like NeuralDEM. The open problems are the ones my thesis met: conservation, generalization to new regimes, and trust, i.e. knowing when the surrogate is out of its data.

11 ML: Advanced Techniques (Brandstetter, Holzleitner)

This chapter follows the three written exams of the course (July 2024, July 2026, September retry). The course has two halves: generative models and AI for physics (Prof. Brandstetter) and statistical theory (Dr. Holzleitner: estimators, learning theory, kernels, EM, causality). If one of them sits on your committee, expect the conversation to start here.

11.1 Generative models and AI for physics (Brandstetter)

2-minute summary ▪ Generative models in two minutes

VAE: encoder $q_\phi(\mathbf{z}|\mathbf{x})$, decoder $p_\theta(\mathbf{x}|\mathbf{z})$, maximize the ELBO $= \mathbb{E}_q[\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})) \le \log p(\mathbf{x})$; reparametrization $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ makes sampling differentiable. Diffusion: forward $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\mathbf{x}_{t-1}, (1-\alpha_t)I)$, closed form $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar\alpha_t}\mathbf{x}_0, (1-\bar\alpha_t)I)$, $\bar\alpha_t = \prod_{s\le t}\alpha_s$; learn to denoise. Flow matching: regress a velocity field, $\|\mathbf{v}_\theta(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2$. Neural ODE: $\mathrm{d}\mathbf{h}/\mathrm{d}t = f_\theta(\mathbf{h}, t)$; a ResNet block is one Euler step. pPCA: $\mathbf{x} = W\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}$, all Gaussian, a linear generative model.

Picture it. Diffusion: drop ink into water and film it (forward process, easy). Then learn to play the film backwards (reverse process, hard): from a cloudy glass back to the clean drop. The VAE is a zip file: compress to a short code, unzip again, and keep the codes tidy (KL term) so random codes also unzip into something sensible.

Board drill (write it by hand 3 times). ELBO $= \mathbb{E}_q[\log p(\mathbf{x}|\mathbf{z})] - D_{\mathrm{KL}}(q(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))$ • $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar\alpha_t}\mathbf{x}_0, (1-\bar\alpha_t)I)$ • $\mathbf{x}_t = \sqrt{\bar\alpha_t}\mathbf{x}_0 + \sqrt{1-\bar\alpha_t}\boldsymbol{\epsilon}$ • loss $\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2$ • ResNet $\mathbf{h}_{l+1} = \mathbf{h}_l + f(\mathbf{h}_l)$ $\leftrightarrow$ Euler $\mathbf{h}(t+\Delta t) = \mathbf{h}(t) + \Delta t\, f(\mathbf{h}, t)$.

Q11.1 COURSE EXAM Diffusion: what is $q(\mathbf{x}_t|\mathbf{x}_0)$, and what are the ELBO terms? Brandstetter

From: ML: Advanced Techniques (Brandstetter) · diffusion · exam July 2024, Q1

Say: Each forward step adds Gaussian noise; Gaussians compose, so we can jump to any step directly: $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar\alpha_t}\mathbf{x}_0, (1-\bar\alpha_t)I)$. The ELBO has three parts. $L_1 = \mathbb{E}_q[\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)]$: reconstruction of the data from the first latent step. $L_2 = D_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T))$: how well the forward process reaches the prior, a standard Gaussian; it has no parameters and goes to 0 for large T. $L_3 = \sum_{t\ge 2}\mathbb{E}_q[D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\|p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))]$: the denoising terms; minimized when the learned denoising step matches the true one, and computed in closed form because both are Gaussian.

Follow-up: "Why predict the noise?" The KL between Gaussians reduces to a squared error between the true and predicted noise: $\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\|^2$.

Source: ML: Advanced Techniques, exam 2024 Q1; `VDM.pdf`.

Q11.2 COURSE EXAM Hierarchical VAE with two latents: posterior and ELBO? Brandstetter

From: ML: Advanced Techniques (Brandstetter) · hierarchical VAE · exam July 2024

Say: Generative chain $\mathbf{z}_2 \to \mathbf{z}_1 \to \mathbf{x}$, so the inference goes the other way: $Q(\mathbf{z}_1, \mathbf{z}_2|\mathbf{x}) = q(\mathbf{z}_1|\mathbf{x})\, q(\mathbf{z}_2|\mathbf{z}_1)$.

Write:

$\text{ELBO} = \mathbb{E}[\log p(\mathbf{x}|\mathbf{z}_1)] - D_{\text{KL}}(q(\mathbf{z}_1|\mathbf{x})\|p(\mathbf{z}_1|\mathbf{z}_2)) - D_{\text{KL}}(q(\mathbf{z}_2|\mathbf{z}_1)\|p(\mathbf{z}_2))$.

Follow-up: "Why is training hard?" Sampling in the forward pass blocks gradients (fixed by the reparametrization trick), and posterior collapse: the KL term can switch latents off.

Q11.3 COURSE EXAM VAE facts from the exam. Brandstetter

From: ML: Advanced Techniques (Brandstetter) · VAE · exam July 2024

Say: Maximizing the ELBO maximizes a lower bound on $\log p(\mathbf{x})$; it is tight, i.e. equal to the log-likelihood, only when $q(\mathbf{z}|\mathbf{x})$ equals the true posterior. We do not know the true posterior, so we approximate it with the encoder. With the reparametrization trick $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ the randomness moves into $\boldsymbol{\epsilon}$ and gradients flow through $\boldsymbol{\mu}, \boldsymbol{\sigma}$. The encoder is not used for generation: sample $\mathbf{z} \sim p(\mathbf{z})$ and decode.

Q11.4 COURSE EXAM Neural ODEs: which statements are true? Brandstetter

From: ML: Advanced Techniques (Brandstetter) · Neural ODEs · exam July 2024

Say: A residual update $\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t)$ is an Euler step; in the limit the forward pass is $\mathrm{d}\mathbf{h}/\mathrm{d}t = f_\theta(\mathbf{h}(t), t)$, solved by any ODE solver, and time plays the role of the layer index. Gradients come from the adjoint method: a second ODE solved backwards in time, whose initial condition is the gradient of the loss at the final state, $\mathbf{a}(T) = \partial L/\partial \mathbf{h}(T)$. Memory is constant in depth because intermediate states are not stored. The backward solve need not use the same time steps as the forward one.

Write:

$$\frac{\mathrm{d}\mathbf{a}}{\mathrm{d}t} = -\mathbf{a}^\top \frac{\partial f}{\partial \mathbf{h}}, \quad \frac{\mathrm{d}L}{\mathrm{d}\theta} = -\int_T^0 \mathbf{a}^\top \frac{\partial f}{\partial \theta}\, \mathrm{d}t.$$

→ hook: *My thesis is also an ODE in time, solved with a very cheap stored operator instead of a learned f.*

Q11.5 COURSE EXAM Describe the Kuramoto–Sivashinsky equation. Why is it used? Brandstetter

From: ML: Advanced Techniques (Brandstetter) · neural PDE surrogates · exam July 2024

Say: A time-dependent, fourth-order, non-linear PDE that produces spatio-temporal chaos: $u_t + uu_x + u_{xx} + u_{xxxx} = 0$. The u_{xx} term pumps energy in at large scales, the fourth-order term damps small scales, the non-linear term moves energy between them. It is a standard benchmark for neural PDE surrogates, because long rollouts must stay stable although errors grow.

> Picture it. A pot of soup on the stove that never settles into a pattern but never blows up either.

Q11.6 COURSE EXAM Probabilistic PCA. Brandstetter

From: ML: Advanced Techniques (Brandstetter) · probabilistic PCA · exam July 2024

Say: A linear Gaussian latent model: $\mathbf{z} \sim \mathcal{N}(0, I)$, $\mathbf{x} = W\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2 I)$; W maps the low-dimensional latent space to data space. Generation: sample $\mathbf{z}$, then sample $\mathbf{x}$ from an isotropic Gaussian around $W\mathbf{z} + \boldsymbol{\mu}$. The maximum-likelihood W is spanned by the eigenvectors with the *largest* eigenvalues of the data covariance, and the likelihood is tractable in closed form. It is the simplest *linear* generative model, and a VAE is its non-linear version.

11.2 Statistical theory (Holzleitner)

2-minute summary ▪ Statistics and learning theory in two minutes

Estimator quality: bias $= \mathbb{E}[\hat{w}] - w$, MSE $=$ bias^2 $+$ Var. Fisher information $I(\theta) = \mathbb{E}[(\partial_\theta \log p(X;\theta))^2]$; Cramér–Rao: any unbiased estimator has $\mathrm{Var}(\hat{\theta}) \geq 1/(nI(\theta))$; the MLE is consistent and asymptotically normal with exactly that variance (efficient). Learning theory (0–1 loss): Bayes classifier (best possible), best model in the class, learned model. Excess risk = approximation error (class too small, deterministic) + estimation error (too little data, random). Hoeffding bounds the gap for *one* fixed model; for the learned model we need *uniform* convergence over the class, via symmetrization and the shattering coefficient, which is polynomial in n when the VC dimension is finite. Kernels: k is a kernel iff every Gram matrix is symmetric positive semi-definite; the RKHS has the reproducing property $f(\mathbf{x}) = \langle f, k(\cdot, \mathbf{x})\rangle$. EM: $\log p(X|\theta) = \mathcal{L}(q,\theta) + D_{\mathrm{KL}}(q\|p(Z|X,\theta))$; E-step sets q to the posterior, M-step maximizes $\mathcal{L}$ over θ. Causality: association $\rightarrow$ intervention (do) $\rightarrow$ counterfactual.

Picture it. Bias is a scale that always shows 1 kg too much; variance is a scale that jumps around. A good scale needs both small, which is exactly the MSE. Learning theory asks: if the scale was right on the 50 items in the shop, will it be right on every item? Hoeffding answers for one scale, uniform convergence for a whole shelf of scales at once.

Board drill (write it by hand 3 times). MSE $=$ bias^2 $+$ Var • $\mathrm{Var}(\hat{\theta}) \geq \dfrac{1}{nI(\theta)}$ • Hoeffding $P(|R_{\mathrm{emp}}(f) - R(f)| > \epsilon) \leq 2e^{-2n\epsilon^2}$ • $R(f) \leq R_{\mathrm{emp}}(f) + \sqrt{\dfrac{h(\log(2n/h)+1) + \log(4/\delta)}{n}}$ • $\gamma_{nk} = \dfrac{\pi_k \mathcal{N}(\mathbf{x}_n|\boldsymbol{\mu}_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_n|\boldsymbol{\mu}_j, \Sigma_j)}$, $\pi_k = N_k/N$.

Q11.7 COURSE EXAM Two estimators of a mean: bias, variance, which to choose? Holzleitner

From: ML: Advanced Techniques (Holzleitner) · estimators · exams 2024 and 2026

Say: The exam example: X_1, X_2, X_3 i.i.d. with $\mu = 3$, $\sigma^2 = 2$; $\hat{w}_1 = \frac{1}{3}(X_1 + X_2 + X_3) + 1$ and $\hat{w}_2 = \frac{1}{3}(X_1 + 2X_2 + X_3)$. Both are biased: $\mathbb{E}\hat{w}_1 = \mu + 1$, $\mathbb{E}\hat{w}_2 = \frac{4}{3}\mu = 4$, so both have bias 1. $\mathrm{Var}(\hat{w}_1) = \frac{3\sigma^2}{9} = 0.67$, $\mathrm{Var}(\hat{w}_2) = \frac{6\sigma^2}{9} = 1.33$. MSE $= 1 + 0.67 = 1.67$ against $1 + 1.33 = 2.33$: choose $\hat{w}_1$.

Follow-up: Rule: $\mathrm{Var}(\sum a_i X_i) = \sigma^2 \sum a_i^2$ for independent X_i; the constant $+1$ changes the bias, not the variance.

Q11.8 COURSE EXAM Fisher information, Cramér–Rao and the MLE. Holzleitner

From: ML: Advanced Techniques (Holzleitner) · Fisher information · exams 2024, 2026, retry

Say: The score $\partial_\theta \log p(X;\theta)$ is the slope of the log-likelihood; its mean is 0 and its variance is the Fisher information $I(\theta)$: how sharply the data pin down θ. Cramér–Rao: every *unbiased* estimator has variance at least $1/(nI(\theta))$. The MLE is consistent and asymptotically normal, $\sqrt{n}(\hat{\theta}_{\mathrm{MLE}} - \theta) \rightarrow \mathcal{N}(0, 1/I(\theta))$, so it reaches the bound asymptotically. The bound says nothing about biased estimators, which can have smaller variance.

Write:

$I(\theta) = \mathbb{E}[(\partial_\theta \log p)^2] = -\mathbb{E}[\partial_\theta^2 \log p]$.

Q11.9 COURSE EXAM Consistency, uniform convergence, Hoeffding, symmetrization. Holzleitner

From: ML: Advanced Techniques (Holzleitner) · statistical learning theory · exams 2024 and 2026

Say: For a *fixed* model the law of large numbers gives $R_{\text{emp}} \to R$, and Hoeffding gives the rate. But the learned model is chosen *with* the data, so we need the gap to be small for all models of the class at once: uniform convergence, which implies consistency (not the other way round in general). Symmetrization replaces the true risk by the empirical risk on a second "ghost" sample, so only the finitely many labellings of $2n$ points matter; their number is the shattering coefficient, polynomial in n if the VC dimension is finite. The bound has the form $R \leq R_{\text{emp}}+$ capacity term. SLT assumes no knowledge of the data distribution.

Q11.10 COURSE EXAM VC dimension of origin-centred circles (inside = +1). Holzleitner

From: ML: Advanced Techniques (Holzleitner) · VC dimension · exams 2024 and 2026

Say: One point: both labels are possible by choosing the radius, so it is shattered, $h \geq 1$. Two points at distances $d_1 < d_2$: the labelling "inner point -1, outer point $+1$" is impossible, because any circle containing the outer point also contains the inner one. So $h = 1$. A class with infinitely many models can still have a small VC dimension.

Q11.11 COURSE EXAM Kernels and RKHS. Holzleitner

From: ML: Advanced Techniques (Holzleitner) · kernels, RKHS · exam July 2026

Say: k is a kernel iff all its Gram matrices are symmetric positive semi-definite (non-negative eigenvalues). Moore–Aronszajn: every such kernel defines a unique RKHS, with the reproducing property $f(\mathbf{x}) = \langle f, k(\cdot, \mathbf{x})\rangle_{\mathcal{H}}$. We never need the feature map explicitly; we only run linear methods in the RKHS through kernel evaluations. Gauss kernel $\exp(-\|\mathbf{x}-\mathbf{x}'\|^2/2\sigma^2)$: similarity decays smoothly with distance; σ is a hyperparameter.

Q11.12 COURSE EXAM EM for mixture models, and maximum entropy. Holzleitner

From: ML: Advanced Techniques (Holzleitner) · EM, maximum entropy · exam 2026, retry

Say: The log of a sum makes the mixture likelihood hard. Introduce one-hot latents $\mathbf{z}_n$. E-step: with the parameters fixed, compute the responsibilities γ_{nk}, the posterior probability that point n belongs to component k (the expectation of the latent). M-step: with the responsibilities fixed, update the parameters; the weights $\pi_k = N_k/N$ come from a Lagrange multiplier for $\sum_k \pi_k = 1$. The likelihood never decreases, but EM finds a local maximum. In general $\log p = \mathcal{L}(q,\theta) + \text{KL}$, and since $\text{KL} \geq 0$, $\mathcal{L}$ is a lower bound. Maximum entropy: among all distributions that match measured moments (e.g. mean and variance), pick the one with the largest entropy, the least extra assumption; mean and variance give the Gaussian. It does not assume a parametric family in advance.

Q11.13 COURSE EXAM Causality: SCM, intervention, backdoor, ladder. Holzleitner

From: ML: Advanced Techniques (Holzleitner) · causality · exam July 2026

Say: A structural causal model writes each variable as a function of its parents and a noise term. An intervention $do(X = x)$ cuts all arrows *into* X, so after intervening on B it is independent of its parent A. The backdoor criterion: a set Z that blocks every path from T to Y entering T from behind, and contains no descendant of T, allows $P(y|do(t)) = \sum_z P(y|t,z)P(z)$. Pearl's ladder: association, intervention, counterfactual. Counterfactuals ("what if Forrest had slept 8 hours?"): keep the individual's noise terms, change the variable, recompute. Randomized controlled trials remove confounding by design.

Write:

Adjustment: $P(y \mid do(t)) = \sum_z P(y \mid t, z)\, P(z)$.

12 New 2025 courses: Geometric DL, Unsupervised, DL Architectures

The 2025 curriculum added *Deep Learning: Geometric Techniques* and renamed the deep learning courses (*Basic Techniques* = old DLNN I, *Architectures and Generative Techniques* = old DLNN II). An examiner who teaches them may use their vocabulary, so this chapter covers what is new. Most of DL Basic is already in Chapters 1–3.

12.1 Deep Learning: Geometric Techniques

2-minute summary ▪ Geometric deep learning in two minutes

Problem: the curse of dimensionality; local methods need exponentially many samples in high dimension. Remedy: priors about the *structure* of the data. Two priors. (1) *Symmetry*: the task does not change under a group G of transformations (translations for images, permutations for sets and graphs, rotations for molecules). (2) *Scale separation*: coarse-grain the signal step by step. Invariance $f(\rho(g)x) = f(x)$, equivariance $f(\rho(g)x) = \rho'(g)f(x)$. Blueprint: equivariant layers → non-linearity → local pooling (coarsening) → repeat → global invariant pooling → MLP. CNNs (grids, translations), DeepSets (sets, permutations), GNNs (graphs, permutations), Transformers (complete graphs) are all instances. Message passing: $\mathbf{h}_i' = \phi(\mathbf{h}_i, \bigoplus_{j\in\mathcal{N}(i)} \psi(\mathbf{h}_i, \mathbf{h}_j, \mathbf{e}_{ij}))$, $\bigoplus$ permutation-invariant (sum, mean, max).

Picture it. A face is still a face if you move the photo (translation) and a molecule is still the same molecule if you number its atoms differently (permutation). Building these facts into the network means it never has to learn them from data, which is the cure for the curse of dimensionality.

Board drill (write it by hand 3 times). Group axioms: closure, associativity, identity, inverse • equivariance $f(\rho(g)x) = \rho'(g)f(x)$ • DeepSets $f(X) = \rho(\sum_i \phi(\mathbf{x}_i))$ • set permutation $X \mapsto PX$ • GCN $H' = \sigma(\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}HW)$ • group averaging $\bar{f}(x) = \frac{1}{|G|}\sum_{g\in G} f(\rho(g)x)$.

Q12.1 COURSE EXAM What is the curse of dimensionality, and what is the manifold hypothesis? GDL lecturer, Klambauer

From: DL: Geometric Techniques · curse of dimensionality · course exam question map

Say: In high dimension the volume grows so fast that data become sparse; local methods (nearest neighbours, kernels with small bandwidth) need a number of samples exponential in the dimension. For Lipschitz functions the estimation error decays only like $N^{-1/d}$. The manifold hypothesis says real data lie near a low-dimensional manifold inside the high-dimensional space, so the *intrinsic* dimension is small; together with symmetry priors this makes learning possible.

Q12.2 COURSE EXAM Define a group, invariance and equivariance. GDL lecturer

From: DL: Geometric Techniques · geometric priors · course exam question map

Say: A group is a set with an operation that is closed and associative, has an identity, and has an inverse for every element. It acts on the domain, and the action lifts to signals via a representation $\rho(g)$ (a linear map). f is invariant if $f(\rho(g)x) = f(x)$, equivariant if $f(\rho(g)x) = \rho'(g)f(x)$: transform then map equals map then transform (a commutative diagram). Inverses are needed so that the lifted action $(\rho(g)x)(u) = x(g^{-1}u)$ is well defined.

Follow-up: "How do you make any function invariant?" Average over the group: $\bar{f}(x) = \frac{1}{|G|}\sum_g f(\rho(g)x)$. The approximation error does not grow if the target is invariant; the estimation error shrinks, roughly by the size of the group.

Q12.3 COURSE EXAM **DeepSets and linear equivariant maps on sets.** GDL lecturer, Klambauer

From: DL: Geometric Techniques · sets · course exam question map

Say: The symmetry group of a set of n elements is the permutation group S_n; a permutation acts on the feature matrix as $X \mapsto PX$. Every linear permutation-equivariant layer has the form $F(X) = XA + \frac{1}{n}\mathbf{1}\mathbf{1}^\top XB$ (the same map for every element plus a shared term). DeepSets: an equivariant per-element encoder ϕ (learned), a sum (fixed, invariant), then a learned decoder ρ: $f(X) = \rho(\sum_i \phi(\mathbf{x}_i))$.

Q12.4 COURSE EXAM **Message passing, and the Weisfeiler–Leman test.** GDL lecturer, Klambauer

From: DL: Geometric Techniques · graphs; also asked (GNNs) in past defenses

Say: A graph adds edges to a set. Each layer: every node computes messages from its neighbours, aggregates them with a permutation-invariant function, and updates its state. Node outputs are equivariant, a final readout makes graph outputs invariant. The WL test refines node colours by hashing each node's colour with the multiset of its neighbours' colours; if the histograms differ, the graphs are not isomorphic; if they agree, we cannot tell. MPNNs are at most as powerful as WL, and exactly as powerful if the aggregation is injective on multisets (sum, as in GIN, not mean or max).

Follow-up: "Limits?" Over-squashing: information from many distant nodes is squeezed through few edges; it depends mostly on graph topology (bottlenecks). Remedies: positional encodings, subgraph GNNs, graph rewiring.

Q12.5 COURSE EXAM **Transformers as graph networks; CNNs as the grid case.** GDL lecturer

From: DL: Geometric Techniques · graphs, grids · course exam question map

Say: Self-attention is message passing on the complete graph with attention weights as edge weights. Without positional encodings it is permutation-equivariant; with them it is not, by design. A CNN is the blueprint on a grid with the translation group: convolution is the (only) linear translation-equivariant map, pooling does the coarse-graining.

12.2 ML: Unsupervised Techniques

2-minute summary ▪ Unsupervised learning in two minutes

PCA: directions of maximal variance = eigenvectors of the covariance; unique up to sign if eigenvalues differ. Kernel PCA: PCA on the centred Gram matrix $\tilde{K} = K - \mathbf{1}_N K - K\mathbf{1}_N + \mathbf{1}_N K \mathbf{1}_N$. ICA: find *statistically independent* non-Gaussian sources; unique up to order and scale; whitening then rotation; contrast functions (kurtosis, negentropy); Infomax. Factor analysis: $\mathbf{x} = W\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}$ with diagonal (not isotropic) noise, so $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, WW^\top + \Psi)$; fitted with EM. MDS: embed points so distances are kept. NMF: $X \approx WH$, $W, H \geq 0$, parts-based. Mixtures + EM, clustering (k-means = hard EM), max entropy.

Picture it. PCA finds the longest axis of the cloud of points. ICA is the cocktail party: two people talk at once into two microphones, and ICA separates the two voices because each voice on its own is not Gaussian.

Board drill (write it by hand 3 times). PCA: $C = \frac{1}{N}\sum_n (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top$, $C\mathbf{u} = \lambda\mathbf{u}$ • FA: $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, WW^\top + \Psi)$ • ICA: $\mathbf{x} = A\mathbf{s}$, $\mathbf{y} = W\mathbf{x}$, maximize non-Gaussianity • k-means: $\min \sum_n \|\mathbf{x}_n - \boldsymbol{\mu}_{c(n)}\|^2$.

Q12.6 COURSE EXAM What is PCA, how is it computed, is it unique? — Klambauer, Hochreiter

From: ML: Unsupervised Techniques · PCA · Fragenkatalog Q8–10

Say: Centre the data, compute the covariance matrix, and take its eigenvectors with the largest eigenvalues (or the SVD of the data matrix). They are the orthogonal directions of maximal variance, and projecting on them gives the best linear reconstruction in squared error. Unique up to the sign of each vector when the eigenvalues are distinct; with equal eigenvalues any rotation inside that eigenspace is also a solution. Properties: linear, orthogonal, uncorrelated components.

Q12.7 COURSE EXAM ICA versus PCA versus factor analysis. — Klambauer, Hochreiter

From: ML: Unsupervised Techniques · ICA, FA · Fragenkatalog Q13–23

Say: PCA: uncorrelated, orthogonal directions of maximal variance. ICA: statistically *independent* components, which needs non-Gaussian sources; unique up to permutation and scaling. It whitens the data (then only a rotation is left) and finds the rotation that maximizes non-Gaussianity with a contrast function such as kurtosis or negentropy; Infomax maximizes the information flow through a non-linearity. Factor analysis: a latent Gaussian model like pPCA but with a diagonal noise covariance, so every feature has its own noise; fitted with EM, whose E-step estimates the posterior mean and covariance of the factors.

Q12.8 PREDICTED Clustering: k-means and its relation to EM. — Klambauer

From: ML: Unsupervised Techniques · clustering, mixtures · predicted

Say: k-means alternates two steps: assign each point to the nearest centre, then move each centre to the mean of its points; the objective $\sum_n \|\mathbf{x}_n - \boldsymbol{\mu}_{c(n)}\|^2$ never increases. It is the hard-assignment limit of EM for a Gaussian mixture with equal isotropic variances: responsibilities become 0 or 1.

12.3 Deep Learning: Architectures and Generative Techniques

2-minute summary ▪ DL Architectures in two minutes

CNN families: LeNet → AlexNet → VGG (small 3×3 filters) → Inception (parallel filters) → ResNet (skips) → DenseNet. Vision tasks: classification; semantic segmentation (class per pixel, U-Net, FCN); object detection (boxes: R-CNN family two-stage, YOLO/SSD one-stage, IoU, non-maximum suppression); instance segmentation (Mask R-CNN). Generative: autoencoder, VAE (ELBO), GAN (minimax), normalizing flows (exact likelihood by change of variables), diffusion, autoregressive models (PixelCNN). $\min_G \max_D \; \mathbb{E}[\log D(\mathbf{x})] + \mathbb{E}[\log(1 - D(G(\mathbf{z})))]$; flow: $\log p(\mathbf{x}) = \log p(\mathbf{z}) + \log|\det \partial \mathbf{z}/\partial \mathbf{x}|$.

Picture it. GAN: a forger (generator) and a detective (discriminator) train against each other until the detective can no longer tell the fakes from the real paintings.

Q12.9 PREDICTED Semantic segmentation versus object detection. Name architectures. — Klambauer

From: DL: Architectures (DLNN II) · segmentation and detection, SS25 script PDF p. 197–239; U-Net asked in a past defense

Say: Semantic segmentation gives every pixel a class; U-Net (encoder–decoder with skip connections) or FCN. Object detection gives boxes plus classes; two-stage detectors (Faster R-CNN: region proposals, then classification) or one-stage (YOLO, SSD: predict boxes on a grid directly). Quality by intersection over union (IoU); duplicate boxes removed by non-maximum suppression. Instance segmentation (Mask R-CNN) separates individual objects of the same class.

Q12.10 PREDICTED GANs: objective, problems, fixes. Klambauer

From: DL: Architectures (DLNN II) · GANs, SS25 script PDF p. 265–279 · predicted

Say: The generator maps noise to samples, the discriminator tells real from fake; they play $\min_G \max_D \mathbb{E}[\log D(\mathbf{x})] + \mathbb{E}[\log(1 - D(G(\mathbf{z})))]$. At the optimum the generator minimizes the Jensen–Shannon divergence. Problems: mode collapse, unstable training, vanishing generator gradients. Fixes: non-saturating loss, Wasserstein GAN, spectral normalization, and the two time-scale update rule (TTUR, from Linz), with the FID to measure quality.

Q12.11 PREDICTED Autoencoder, VAE, normalizing flow: what is the difference? Klambauer

From: DL: Architectures (DLNN II) · generative models, SS25 script PDF p. 242–297 · predicted

Say: An autoencoder compresses and reconstructs; its latent space is not organised, so it is poor for sampling. A VAE makes the latent space a distribution with a KL term, so we can sample, but the likelihood is only bounded (ELBO). A normalizing flow uses invertible layers and the change of variables formula, so the likelihood is exact, at the price of equal dimensions and constrained layers. Diffusion trades speed for quality with many denoising steps.

13 October 2026: the newest questions, derived step by step

2-minute summary ▪ What was asked in October 2026

Two committees: Hoedt, Schlüter, Zoitl and Mayr, Schlüter, Luukkonen, and one with Zoitl, Hochreiter, Kobler. Schlüter: what is deep learning, what is a layer, linear activations, sigmoid versus step function, learning rate and batch size, SGD versus Adam, receptive field and pooling. Hochreiter: delta errors, vanishing gradients and their fixes, initialization and its theory, the spectral norm and the structure of W, regularization, optimizers, activations, normalizations (RMSNorm), AdamW versus Adam with L2, the SVM primal and the meaning of the "1". Rule for every answer: one sentence to learn by heart, then derive the formula on the board.

Q13.1 ASKED BEFORE What is deep learning? What is a layer, and what happens in a layer? Schlüter

From: DL Basic Techniques (DLNN I) · Ch. 2 Neural networks · asked October 2026

Say: Deep learning trains a network of many layers end to end, and every layer turns the representation of the layer below into a more useful one. Inside a layer there are two steps: a linear map, then a non-linear activation.

Write:

One neuron sums its weighted inputs, adds a bias and bends the result. Stacking the weight rows of all neurons of a layer gives a matrix:

$$s_i = \sum_j w_{ij} a_j + b_i \implies \mathbf{s} = W\mathbf{a} + \mathbf{b}, \qquad \mathbf{a}^{\text{new}} = f(\mathbf{s}).$$

Picture it. A layer is "linear, then bend". Deep means features built from features: edges, then parts, then objects.

Q13.2 ASKED BEFORE Can an activation function be linear? Schlüter

From: DL Basic Techniques (DLNN I) · Ch. 2 · asked October 2026

Say: At the output, yes, for regression. In the hidden layers, no, because linear times linear stays linear.

Write:

Two layers without activation collapse into one:

$$W_2(W_1\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2 = \underbrace{W_2W_1}_{W'}\mathbf{x} + \underbrace{W_2\mathbf{b}_1+\mathbf{b}_2}_{\mathbf{b}'}.$$

So any depth gives one linear model, which cannot solve XOR. For regression the target is any real number, so the output stays linear.

Q13.3 ASKED BEFORE Supervised or unsupervised: what is the difference, and why choose one? Defense

From: ML: Supervised Techniques (Lehner) unit 1 · asked October 2026

Say: Supervised learning has a label for every input and learns the map from input to label, unsupervised learning has no labels and looks for structure in the inputs.

Write:

Supervised: data $\{(\mathbf{x}_n, y_n)\}$, learn $g(\mathbf{x}) \approx y$ (classification, regression). Unsupervised: data $\{\mathbf{x}_n\}$, learn clusters, a density $p(\mathbf{x})$ or a projection (PCA). Self-supervised: labels made from the data, for example the next word.

Follow-up: "Why did you choose it?" I had a clear target and labelled data. Without labels, or to explore the data first, I would choose unsupervised.

Q13.4 ASKED BEFORE Binary classification with labels 0 and 1: what is the output, why a sigmoid, and could we use a step function? Schlüter

From: DL Basic Techniques (DLNN I) · Ch. 4 Loss functions, Ch. 6 Backprop · asked October 2026

Say: The output is the probability of class 1. The sigmoid gives a smooth value in $(0,1)$, and with cross-entropy its delta is prediction minus label. A step function has zero derivative almost everywhere, so every delta is zero and nothing is learned.

Write:

Step 1, the slope of the sigmoid:

$$\sigma(s)=\frac{1}{1+e^{-s}}, \qquad \sigma'(s)=\frac{e^{-s}}{(1+e^{-s})^2}=\sigma(s)(1-\sigma(s))\le\tfrac{1}{4}.$$

Step 2, binary cross-entropy and its derivative:

$$L=-\big[y\log a+(1-y)\log(1-a)\big], \qquad \frac{\partial L}{\partial a}=\frac{a-y}{a(1-a)}.$$

Step 3, the chain rule. The factors $a(1-a)$ cancel:

$$\delta=\frac{\partial L}{\partial a}\,\sigma'(s)=a-y.$$

Step function: $H'(s)=0$ for $s\neq 0$ and undefined at 0, so $\delta=0$. The perceptron used a step, but with its own rule, not backpropagation.

Picture it. A flat staircase: standing on a step you cannot feel which way is down.

Q13.5 ASKED BEFORE How does the learning rate influence training? Why small, why large? Schlüter

From: DL Basic Techniques (DLNN I) · Ch. 7 Optimization · asked October 2026

Say: The learning rate is the step size: too small and training crawls, too large and the loss oscillates or diverges.

Write:

Near a minimum the loss along one direction is a parabola with curvature h:

$$L(w) = \tfrac{1}{2}hw^2, \quad \nabla L = hw, \quad w^{\text{new}} = w - \eta h w = (1 - \eta h)\, w.$$

Each step multiplies the distance by $1 - \eta h$. Convergence needs $|1 - \eta h| < 1$, so

$$0 < \eta < \frac{2}{h}.$$

Tiny η: factor close to 1, slow. $\eta > 2/h$: factor above 1 in size, divergence. In practice: warm-up, then a decay (cosine).

Q13.6 ASKED BEFORE Learning rate versus batch size: how are they related? Schlüter

From: DL Basic Techniques (DLNN I) · Ch. 7 · asked October 2026

Say: A bigger batch gives a less noisy gradient, so it can take a bigger step: the linear scaling rule multiplies the learning rate by the same factor as the batch size.

Write:

Mini-batch gradient: unbiased, with variance divided by B:

$$\mathbb{E}[\mathbf{g}_B] = \nabla R_{\text{emp}}, \qquad \text{Var}[\mathbf{g}_B] = \frac{\sigma^2}{B}.$$

k small steps with batch B move about $\eta \sum_{t=1}^{k} \mathbf{g}_B^{(t)} \approx k\eta\, \mathbf{g}_{kB}$ (if the gradient changes little), which is one step with batch kB and rate $k\eta$. So

$$B \to kB \quad \Longrightarrow \quad \eta \to k\eta$$

(Goyal et al. 2017, with a warm-up). The noise level is set by η/B. Very large batches stop paying off.

Q13.7 ASKED BEFORE Gradient descent versus Adam on mini-batches: how does each one use the data? Schlüter

From: DL Basic Techniques (DLNN I) · Ch. 7 Adam · asked October 2026

Say: Both take one step per mini-batch, but SGD uses only the current gradient with one global rate, while Adam keeps running averages of past gradients and gives every weight its own step size.

Write:

Adam, with $\mathbf{g}$ the gradient of the current batch:

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1)\mathbf{g}, \qquad \mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1 - \beta_2)\mathbf{g}^2.$$

Bias correction: both start at zero. Unrolling gives $\mathbb{E}[\mathbf{m}_t] = (1 - \beta_1^t)\, \mathbb{E}[\mathbf{g}]$, so

$$\hat{\mathbf{m}} = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}} = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad \mathbf{w} \leftarrow \mathbf{w} - \eta\, \frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{v}}} + \epsilon}.$$

Scale invariance: $\mathbf{g} \to c\,\mathbf{g}$ gives $\hat{\mathbf{m}} \to c\,\hat{\mathbf{m}}$ and $\sqrt{\hat{\mathbf{v}}} \to c\sqrt{\hat{\mathbf{v}}}$, so the step is unchanged.

Picture it. SGD forgets the last batch. Adam remembers.

Q13.8 ASKED BEFORE What is the difference between AdamW and Adam with L2 regularization? Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 7–8 · asked October 2026

Say: AdamW applies the weight decay directly to the weights, outside the adaptive division. Adam with L2 sends the decay through the division, so weights with large gradients are hardly decayed.
Write:
For SGD the two are the same:

$$\mathbf{w} - \eta(\mathbf{g} + \lambda\mathbf{w}) = (1 - \eta\lambda)\,\mathbf{w} - \eta\,\mathbf{g}.$$

Adam with L2 uses $\mathbf{g} + \lambda\mathbf{w}$ inside the moments, so the decay is divided by $\sqrt{\hat{\mathbf{v}}}$. AdamW (Loshchilov and Hutter 2019) keeps it outside:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\Big(\frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{v}}} + \epsilon} + \lambda\mathbf{w}\Big).$$

Q13.9 ASKED BEFORE Which normalizations do you know, and which one is popular now? Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 10 Normalization · asked October 2026
Say: Batch norm normalizes each feature over the batch, layer norm normalizes over the features of one sample, and RMSNorm, the popular one in large language models, only divides by the root mean square.
Write:
One recipe for a set of numbers $x_1, \ldots, x_d$:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad y_i = \gamma\hat{x}_i + \beta.$$

Batch norm: the set is one feature over the batch. Layer norm: all features of one sample. RMSNorm (Zhang and Sennrich 2019) drops the mean:

$$y_i = \frac{x_i}{\sqrt{\frac{1}{d}\sum_k x_k^2 + \epsilon}}\,g_i.$$

The rescaling, not the re-centring, stabilizes training, and RMSNorm is cheaper (used in LLaMA).

Q13.10 ASKED BEFORE Which initialization techniques do you know? Why this variance, and what is the theory? Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 10 Initialization · asked October 2026
Say: LeCun, Glorot and He choose the weight variance so that the variance of the signal, and of the deltas, stays constant from layer to layer.
Write:
Independent, zero-mean weights, independent of the inputs:

$$s = \sum_{j=1}^{n_{\text{in}}} w_j a_j \implies \operatorname{Var}(s) = n_{\text{in}}\operatorname{Var}(w)\,\mathbb{E}[a^2].$$

Constant variance needs $\operatorname{Var}(w) = 1/n_{\text{in}}$ (LeCun, used with SELU). ReLU gives $\mathbb{E}[a^2] = \frac{1}{2}\operatorname{Var}(s)$, so $\operatorname{Var}(w) = 2/n_{\text{in}}$ (He). The backward pass sums over n_{out}, so Glorot takes the compromise

$$\operatorname{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

Zero initialization fails: all units stay identical (symmetry). Orthogonal initialization sets all singular values to 1.

Q13.11 ASKED BEFORE How is the spectral norm defined? Which structure do we want for W, and how should it change over learning? — Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 6 Vanishing gradient; SS25 theory · asked October 2026

Say: The spectral norm is the largest singular value, the largest factor by which W can stretch a vector. We want W close to an isometry, with singular values near one, and the norm should stay bounded during training.

Write:

Definition and its link to eigenvalues:

$$\|W\|_2 = \max_{\mathbf{x}\neq 0} \frac{\|W\mathbf{x}\|}{\|\mathbf{x}\|}, \qquad \|W\|_2^2 = \max_{\mathbf{x}\neq 0} \frac{\mathbf{x}^\top W^\top W\mathbf{x}}{\mathbf{x}^\top\mathbf{x}} = \lambda_{\max}(W^\top W) = \sigma_{\max}^2.$$

With 1-Lipschitz activations, every layer stretches a difference at most by $\|W_l\|_2$, so

$$\|f(\mathbf{x}) - f(\mathbf{x}')\| \leq \prod_l \|W_l\|_2 \, \|\mathbf{x} - \mathbf{x}'\|.$$

If all $\sigma_i = 1$, W is orthogonal: signals and deltas keep their size. During training the weights learn structure and their norms grow, so we keep them bounded (weight decay, spectral normalization $W/\|W\|_2$).

Q13.12 ASKED BEFORE Explain vanishing gradients. What can we do to mitigate them? — Hochreiter

From: DL Basic Techniques (DLNN I) · Ch. 6; LSTM & Transformers (Hochreiter) · asked October 2026

Say: The delta of an early layer is a product of one factor per layer, so it vanishes or explodes when these factors are below or above one. Every fix keeps the factors close to one.

Write:

Unroll the delta recursion and take norms:

$$\boldsymbol{\delta}^{(l)} = f'(\mathbf{s}^{(l)}) \odot W^{(l+1)\top}\boldsymbol{\delta}^{(l+1)}, \qquad \|\boldsymbol{\delta}^{(1)}\| \leq \prod_l \Big(\max|f'| \, \|W^{(l)}\|_2\Big)\|\boldsymbol{\delta}^{(L)}\|.$$

Sigmoid: $\max|f'| = \frac{1}{4}$, so if $\frac{1}{4}\|W\|_2 < 1$ the delta decays exponentially with depth (Hochreiter 1991). Residual block: $\mathbf{a}^{\text{new}} = \mathbf{a} + F(\mathbf{a})$ gives the Jacobian $I + \partial F/\partial\mathbf{a}$, which does not vanish. Fixes: initialization, ReLU/ELU/SELU, normalization, residual connections, gradient clipping, LSTM.

Q13.13 ASKED BEFORE Write down the SVM primal. What do we optimize, where does the "1" come from, and could we use another value? What is the canonical form? — Hochreiter

From: ML: Supervised Techniques (Lehner) · SVM · asked October 2026

Say: We maximise the margin. The 1 only fixes the free scale of $(\mathbf{w}, b)$, so any positive value gives the same hyperplane. Fixing it to 1 is the canonical form.

Write:

Distance of a point to the hyperplane, and the free scale:

$$d_i = \frac{|\mathbf{w}^\top\mathbf{x}_i + b|}{\|\mathbf{w}\|}, \qquad (\mathbf{w}, b) \to (c\mathbf{w}, cb) \text{ gives the same hyperplane.}$$

Canonical form: $\min_i y_i(\mathbf{w}^\top\mathbf{x}_i + b) = 1$, so the margin is $2/\|\mathbf{w}\|$. Maximising it means

$$\min_{\mathbf{w},b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top\mathbf{x}_i + b) \geq 1 \ \ \forall i.$$

With c instead of 1, put $\mathbf{w}' = \mathbf{w}/c$, $b' = b/c$: the constraint becomes ≥ 1 again. Same classifier.

Q13.14 ASKED BEFORE CNN: what is the receptive field, how can we increase it, and what does pooling do?
Schlüter

From: DL Basic Techniques (DLNN I) · CNN chapter · asked October 2026

Say: The receptive field is the part of the input that can influence one unit. It grows with depth, kernel size, stride or pooling, and dilation.

Write:

A layer with kernel k adds $k-1$ positions, measured in the jump j (the product of earlier strides):

$$r_l = r_{l-1} + (k_l - 1)\, j_{l-1}, \qquad j_l = j_{l-1} s_l, \qquad r_0 = j_0 = 1.$$

Three 3×3 layers, stride 1: $r = 1 + 2 + 2 + 2 = 7$. Stride or pooling 2 doubles j. Dilation d: effective kernel $d(k-1)+1$. Pooling (max or average) has no weights, shrinks the map, enlarges later receptive fields and adds a little shift invariance. Global average pooling gives one vector per image or sequence.